VECTOR



MCAL Integration Package Basics and Workflows

User Manual

virgaj, virgp, virrsu, viriwu

V2.05.03 | Released

Document Information

History

Author	Date	Version	Remarks
Virrsu; viriwu	2015-02-27	1.00.00	Initial Ideas, usage as Application Note; Porting to Technical Reference template; adding detailed description about 3rd party tools etc.
Virgp	2015-04-24	1.00.01	Review; small changes to increase understandability; Known Issue for missing config items added → released
Virgaj; Virrsu	2015-06-30	1.00.02	4.3 / 4.5 - Added known issues regarding EB tresos™ tool
Virgp	2015-07-17	1.01.00	2.2.3 - Introduction of Mixed AUTOSAR use case 2.3 ff - Extend description of MCAL preparation (prerequisites) 2.4 - hint about recommended workflow added
Virgp	2015-10-26	1.01.01	2.3.2.2 / 2.3.2.3 - parameter corrected 2.4 – added hint for MCAL Integration video
Virgp	2016-01-26	1.02.00	2.4.2 - added hint for “round trip” ability general - new CI applied
Virgp	2016-07-05	1.03.00	Page 5 - Added further useful documents as reference
Virgp	2016-09-27	1.04.00	1 – completely new Reference to QuickStart document deleted (replaced within this document); Reference to ScreenCase and ReleaseNote added Hints for AUTOSAR3 <-> AUTOSAR4 differentiation added 2.3.2.2 – non-interactive mode introduced

			2.3.2.3 – reference to Release Notes added for further info – completely new 4 – issue for “MCAL and SIP storage location” removed 5 – hint to “generate all” added
Virrsu	2017-11-09	1.05.00	3 - Adapted chapter: Details regarding 3rd party MCAL Integration Helper 4.2 - Adapted Chapter: "Set User Defined" Action 5.3 - Added chapter 5.5 - Added chapter
Virrsu	2018-04-27	1.05.01	5.3 - Extended chapter
Visem	2018-06-28	2.00.00	Document type changed to User Manual
Virgaj	2019-10-17	2.00.01	Added chapter 3 Delivery without MCAL Integration Package
Virld	2019-11-07	2.00.02	Move Helper Tool chapter into chapter 2.5 Configuration Tools. Add most frequently used use cases
Virrsu	2020-01-17	2.01.00	Complete rework and update
Vismna	2020-07-06	2.01.01	Add Disclaimer
Visem	2021-02-08	2.01.02/03	Cover acc. new CD
Virld	2021-07-09	2.02.00	2.5.1.6 - new chapter describing non-interactive user mode of 3rd party MCAL Integration Helper
Virnid	2022-03-14	2.03.00	Adding information for pre-configuration (chapter 2.5.4)
Virnid	2022-03-23	2.04.00	Updating information for pre-configuration; Adding information for usage of different Workspace (chapter 2.5.4)
Virnid / Virld	2022-06-15	2.05.00	Adding chapters for Tresos NG. Adding information about the new 3rdPartyMcalMemMap tool into chapter 2.5.1.4

Visem Virnid	2022-08-17 2022-10-27	2.05.01	DaVinci Configurator Classic SIP→BSW Package Updating information about PerformanceOpimization (chapter 2.5.4.1)
Virld	2022-11-10	2.05.02	Remove obsolete Helper Tool command --delivery from chapter 2.5.1.6
Virnid	2023-02-16	2.05.03	Update of methodology

Reference Documents

No.	Source	Title	Version
[1]	Vector	Product Information MICROSAR Vector SLP4	Latest
[2]	Vector	Catalog – Product Information MICROSAR – Chapter MCAL	Latest
[3]	Vector	Application Note “AN-ISC-8-1171_Tresos_LicenseHandling.pdf”	Latest (e.g. 1.00.01)
[4]	Vector	Technical Reference "TechnicalReference_3rdParty-MCAL-Integration_<controller>.pdf"	Provided within your BSW Package
[5]	Vector	ScreenCast_McalIntegration_Tresos.pdf	Provided within your BSW Package
[6]	Vector	TechnicalReference_Asr_MemoryMapping.pdf	Provided within your BSW Package
[7]	Vector	TechnicalReference_Cdd_Communication.pdf	Provided within your BSW Package
[8]	Vector	UserManual_3rdParty-MCAL-Memory-Mapping.pdf	Provided within your BSW Package



Caution
This document gives an overview on product-wide behavior or use-cases. To ensure correct operation of the product always follow the instructions provided in the technical references and (additionally for safety projects) the safety manual. The content of these documents always replaces/overrides the content from this user manual.

Contents

1 Purpose of the Document 11

2 Delivery with MCAL Integration Package 12
2.1 Purpose of this Chapter 12
2.2 Introduction 12
2.2.1 Responsibility 13
2.2.2 Support Requests 13
2.2.3 Mix Between AUTOSAR Specification Versions 13
2.3 First Steps 13
2.3.1 Delivery Structure 13
2.3.2 Starting Up 16
2.3.2.1 MCAL Delivered Within Vector BSW Package 16
2.3.2.2 MCAL Not Contained Within Vector BSW Package 16
2.3.2.3 MCAL Update Needed 18
2.4 Workflow 18
2.4.1 Single Configuration Tool Usage 19
2.4.2 Mixed Configuration Tool Usage 20
2.4.3 Split Configuration Tool Usage 23
2.5 Configuration Tools 23
2.5.1 3rd Party MCAL Integration Helper Tool 24
2.5.1.1 First Integration of third party Packages 24
2.5.1.2 Change of Derivative 26
2.5.1.3 Add additional third party Packages 28
2.5.1.4 Update of third party Packages 29
2.5.1.5 Remove all third party Packages 33
2.5.1.6 Using the Non-Interactive Mode 34
2.5.1.6.1 DERIVATIVE 34
2.5.1.6.2 ID 35
2.5.1.6.3 PATH 35
2.5.2 Vector DaVinci Configurator Classic 36
2.5.3 EB tresos™ 36
2.5.3.1 EB tresos™ Classic 36
2.5.3.1.1 Setting up a New Configuration Project 37
2.5.3.1.2 Project Details 37
2.5.3.1.3 Selection of Components 38
2.5.3.1.4 Creation of Importers and Exporters 39
2.5.3.1.5 Configure the MCAL Components 40
2.5.3.1.6 Generation of the third party MCAL 40
2.5.3.2 EB tresos™ Next Generation 40

2.5.3.2.1 Setting up a New Configuration Project 40
2.5.3.2.2 Project Details 41
2.5.3.2.3 Selection of components 42
2.5.3.2.4 Configure and Generate the MCAL Components 43
2.5.4 Cooperation Interface Between DaVinci Configurator Classic and EB tresos™ 43
2.5.4.1 EB tresos™ Classic 44
2.5.4.2 EB tresos™ Next Generation 46
2.5.5 Configuration Hints for Parallel Usage of DaVinci Configurator Classic and EB tresos™ 47
2.5.5.1 Vector DaVinci Configurator Classic 47
2.5.5.2 EB tresos™ 47
2.5.6 DaVinci Configurator Classic Services and Comforts 48
2.5.6.1 Internal Behavior Editor 48
2.5.6.2 Initialization Function Service 49

3 Delivery without MCAL Integration Package 50
3.1 Purpose of this Chapter 50
3.2 Integration in DaVinci Configurator Classic 50
3.2.1 Configuration With DaVinci Configurator Classic 50
3.2.1.1 Adding of a BSWMD File 51
3.2.1.2 Adding a Module to the Current Project 52
3.2.2 External Generation Step 53
3.2.2.1 Manual Setup 53
3.2.2.2 Automatic Setup 54
3.2.3 Internal Behavior Description 54
3.2.3.1 Example of Internal Behavior Description 55
3.2.3.2 Templates for Internal Behavior Description 58
3.2.4 RTE Configuration 58
3.2.5 CDD Configuration 58
3.3 Settings.xml 60
3.4 Integration Into the Build Project 60
3.4.1 Compiler_Cfg.h 61
3.4.2 MemMap.h 61

4 Known Issues 63
4.1 Long Path Names 63
4.2 Missing Configuration Items Within Imported Configuration 63
4.3 EB tresos™ Project Cannot Be Created 64
4.4 Configuration Export With EB tresos™ Version 13.0.0 64
4.5 Error Messages Regarding CommonPublishedInformation With EB tresos™ 65

4.6 Problems Due to Resorting of Configuration Elements by EB tresos™ Exporter 65

5 Frequently Asked Questions 66

6 Glossary and Abbreviations 67

6.1 Glossary 67

6.2 Abbreviations 67

7 Contact 68

Illustrations

Figure 2-1 Configuration Workflow – Mixed configuration tool usage 21
Figure 2-2 Da Vinci Configurator Classic: Set User Defined 22
Figure 2-3 3rdPartyMcalIntegrationHelper: First Integration 25
Figure 2-4 3rdPartyMcalIntegrationHelper: Keep Packages 27
Figure 2-5 3rdPartyMcalIntegrationHelper: Derivative Selection 27
Figure 2-6 3rdPartyMcalIntegrationHelper: Additional Packages 29
Figure 2-7 3rdPartyMcalIntegrationHelper 31
Figure 2-8 3rdPartyMcalIntegrationHelper Package Update 31
Figure 2-9 DaVinci Configurator Classic: Updating Modul Definitions 32
Figure 2-10 3rdPartyMcalIntegrationHelper: Remove Packages 33
Figure 2-2-11 3rdPartyMcalIntegrationHelper.log: Available Derivatives 35
Figure 2-2-12 MIPconfig.xml: Package Id 35
Figure 2-2-13 Command Line: Replace Call 36
Figure 2-14 EB tresos™ User Interface: New Configuration Project 37
Figure 2-15 EB tresos™ User Interface: Configuration Project Data 38
Figure 2-16 EB tresos™ User Interface: Component Configurations 39
Figure 2-17 EB tresos™ User Interface: Create an exporter (step 1) 39
Figure 2-18 EB tresos™ User Interface: Create an exporter (step 2) 39
Figure 2-19 EB tresos™ User Interface: Generate Button 40
Figure 2-20 EB tresos™ User Interface: New Configuration Project 41
Figure 2-21 EB tresos™ User Interface: Set Target of the Project 41
Figure 2-22 EB tresos™ User Interface: Configuration of Package Path 42
Figure 2-23 EB tresos™ User Interface: Setting the Module Configuration Location 43
Figure 2-24 DaVinci Configurator Classic: Project Settings - Generator User Settings 44
Figure 2-25 DaVinci Configurator Classic: Routing of EB tresos™ Generator Messages 44
Figure 2-26 DaVinci Configurator Classic: Generation Result View 44
Figure 2-27 DaVinci Configurator Classic: Project Settings - Generator User Settings 46
Figure 2-28 DaVinci Configurator Classic: DEM Path Using Basic Editor 47
Figure 2-29 DaVinci Configurator Classic: DEM Settings 47
Figure 2-30 EB tresos™: DEM Module Configuration 48
Figure 2-31 EB tresos™: DEM-Path in the Outline Window 48
Figure 2-32 EB tresos™: Settings Within DEM Module Editor 48
Figure 2-33 DaVinci Configurator Classic: Internal Behavior Editor 48
Figure 2-34 DaVinci Configurator Classic: Initialization Service 49
Figure 2-35 DaVinci Configurator Classic: EcuC Editor BswInitilization 49
Figure 3-1 Eparm: Overview of BSWMD file of MCU 51
Figure 3-2 DaVinci Configurator Classic: Modules|Additional Definitions 51
Figure 3-3 DaVinci Configurator Classic: Module Assistant 52
Figure 3-4 DaVinci Configurator Classic: Module Definitions 53
Figure 3-5 DaVinci Configurator Classic: Module is Part of the Project 53
Figure 3-6 DaVinci Configurator Classic: External Generation Steps 54
Figure 3-7 DaVinci Configurator Classic: Module Definitions CDD 59
Figure 3-8 DaVinci Configurator 4: Configuring CDD 59
Figure 4-1 EB tresos™: Resource Restrictions 64

Tables

Table 2-1 Guidance for single configuration tool usage 20
Table 2-2 Guidance for Mixed Configuration Tool Mode 21
Table 2-3 Guidance for Split Configuration Tool Usage 23

Table 2-4 Command Line Options of 3rd Party MCAL Integration Helper Tool 34

1 Purpose of the Document

This document supports the user in integrating third MCAL modules (according AUTOSAR specification) into the Vector BSW Package.

If the MCAL Integration Package is **not** part of the BSW Package, the main integration effort must be spent by the user. Please refer to the related chapter 3 Delivery without MCAL Integration Package for further information.

If the MCAL Integration Package has been delivered with the BSW Package, please follow the notes in chapter 2 Delivery with MCAL Integration Package.

2 Delivery with MCAL Integration Package

2.1 Purpose of this Chapter

This chapter supports the user by launching the delivery, setting up a project and serving the tooling- and configuration-based interfaces between the Vector MICROSAR Classic BSW and the third party MCAL.

The term “tooling” means Vector DaVinci Configurator Classic on the one hand and a third party configuration and generation framework (EB tresos™, KPIT ECU Spectrum™ …) on the other.

2.2 Introduction

The *MICROSAR Classic MCAL Integration Package* covers the integration of a *third party MCAL* package into the Vector MICROSAR Classic BSW stack. As part of this, Vector performs a setup of the MCAL provided by the customer or the semiconductor vendor and supplements it with items needed for the integration with the MICROSAR Classic BSW.

Typically, the semiconductor vendor delivers the MCAL with its own configuration and generation tool chain. Vector provides solutions to deal with the dependencies and interfaces on embedded and on configuration level between the AUTOSAR BSW and the MCAL components (e.g. with AUTOSAR4.0.3 these configuration level-based interfaces are about 10 parameters).

The following use cases and corresponding workflows are explained and illustrated in detail later:

1. *Single configuration tool usage*: Usage of the Vector DaVinci Configurator Classic as exclusive configuration tool for both the MICROSAR Classic BSW and the third party MCAL.
2. *Mixed configuration tool usage*: Parallel and synchronized usage of the DaVinci Configurator Classic and a third party MCAL configuration tool.
3. *Split configuration tool usage*: Split configuration and code generation using DaVinci Configurator Classic for the configuration/generation of the MICROSAR Classic BSW and the third party tool for configuration/generation of the MCAL.

In principal, all three use cases are supported and the decision which one is used depends on user requirements as well as on the capabilities of the integrated MCAL. Detailed decision guidance and Notes will be given in this document.



Note
Many items described within this document are AUTOSAR release independent. Wherever necessary we will add a note when there is a deviation within the workflow or any other topic from AUTOSAR specification.

2.2.1 Responsibility

Vector does not take over any responsibility for the usage and functionality of the integrated third party components. The *MCAL integration* into the MICROSAR Classic BSW Package does not include a full functional test for the third party components. It remains the responsibility of the customer to ensure freeness of defect and shortcomings as well as functional completeness of the third party components. The MCAL vendors usually provide regular issue lists to inform the users about known, fixed and/or new issues. Vector usually documents access to such third party issue lists within the BSW Package.

2.2.2 Support Requests

Vector will support you in all aspects of the *MCAL integration* into your MICROSAR Classic BSW Package. In case of questions or problems please get in touch with your MICROSAR Classic contact (e.g. as defined in the delivery description that is part of your BSW Package). Since Vector is not the vendor of the integrated third party components such support requests shall be directed to the third party software vendor as they can provide you with first-hand information and support.

2.2.3 Mix Between AUTOSAR Specification Versions

Sometimes it is necessary that there is a mixture of components released for different AUTOSAR versions. Most common case of mixture is the use of AUTOSAR 3.x related BSW (because OEMs have fixed their platform SW) with AUTOSAR4.0.x related MCALs. This is since there are continuously new HW platforms available through the semiconductor vendors but not all devices will be supported with AUTOSAR4- as well as AUTOSAR3-based MCALs. In future, there will be also the customer-driven situation of mixture of several AUTOSAR4.x specification from MCAL to BSW or vice versa.
SW based on AUTOSAR mixes in general is not directly compatible. Nevertheless, Vector supports you to be able to mix those components. There are several measures to take and Vector provides a solution for this with the term *MCAL Integration Package (mixed ASR)*. There are some wrappers (on embedded as well as on configuration side) that are part of your BSW Package.



Note
Currently Vector supports mixing of AUTOSAR3.x related BSW with MCALs up to AUTOSAR4.0.3 but no higher releases.

2.3 First Steps

During *MCAL integration* at Vector several supplements have been created and integrated into your delivery. A short overview of contributed parts shall be given.

2.3.1 Delivery Structure

The following parts of the MCAL Integration Package (MIP) within the delivery (MICROSAR Classic BSW Package) are relevant for a successful *MCAL integration (based on an AUTOSAR4 BSW BSW Package)* reflecting legacy / component-based approach starting from BSW Package root path.



Note
The delivery structure may vary in wide parts. The legacy approach of the up-to-now structure has been replaced by a component-based approach, but both are mentioned below. Following description is meant to be a general one. Single artifacts may be located slightly different. The outdated AUTOSAR3 approach based on a different configuration tooling is not reflected here.

Legacy	Component-based	MIP Content
`BSW\Mcal_<µC>`	`Components\Mcal_<µC>\Implementation`	Embedded parts for smooth build process.
`BSW\Mcal_<µC>\mak`	`Components\Mcal_<µC>\Make`	AUTOSAR make-files for all third party MCAL components (typically all MCAL components are merged into one single file).
`BSWMD\Mcal_<µC>`	`Components\Mcal_<µC>\BSWMD`	Configuration items for integration into the DaVinci Configurator Classic. After execution of steps described under 2.3.2 the basic software definition files (VSMD - the third party MCAL specific module definition files) are located here as well as the Vector additions to the module definition files.
`DaVinciConfigurator\Generators\Mcal_<µC>`	`Components\Mcal_<µC>\GeneratorMsr`	Configuration items for integration into the DaVinci Configurator Classic. Those files relate to configuration and generation via Vector tooling, e.g. XML-preparation files as well as files containing the commands for execution of the third party code generator plus additional convenience features like registration of Init-APIs towards the ECUM component.
`Doc\<DocType>`	`Components\Mcal_<µC>\Documentation`	MCAL / µC specific documentation provided by Vector
	`Doc\<DocType>`	MIP specific documentation provided by Vector
`ThirdParty\Mcal_<µC>\Supply`	`ThirdParty\Mcal_<µC>\Supply`	third party packages and the tooling provided along with.

ThirdParty\Mcal_<µC>\VectorIntegration	ThirdParty\Mcal_<µC>\VectorIntegration	Tools that support the user during project setup (refer to 2.3.2)



Note
Typical <DocType> are ApplicationNote, DeliveryInformation, TechnicalReference or UserManual.



Note
The structure of the third party packages are defined by the vendors. Typically, the supply folder contains MCAL drivers, external tool, security drivers, safety SW or other stuff. The mandatory MCAL package might contain:

> MCAL implementation (C- and H files)

> BSWMD description files according to AUTOSAR

> Tooling for MCAL configuration and generation (incl. templates)

> MCAL related documentation



Caution
Your delivery (BSW Package) may not finally contain the integrated MCAL because of several reasons (license etc.). The MCAL and its generation tooling, to be used with Vectors BSW Package, must be installed directly into this above-mentioned folder ThirdParty\<MCAL_µC>\Supply with exactly the default name of your MCALs supplier!



Note
Please consider that the Vector BSW Package contains a GUI-based tool called *3rd party MCAL Integration Helper*. This tool provides some features supporting the user in handling the third party MCAL and preparing it for smooth usage together with the Vector Software stack (refer to 2.3.2). Depending on the deployment of your MCAL supplier (keyword: one package for the whole MCAL, several packages for single functionalities like basic I/O, memory, communication) you will have to install one or more packages into the Supply folder of your BSW Package. The *3rd party MCAL Integration Helper* tool will guide you through this process.

Example with MCAL from third party vendor Infineon:

```
ThirdParty\<MCAL_µC>\Supply
.\MC-ISAR_AS4XX_AURIX_TC27X_CA_PB_BASE_V100
.\MC-ISAR_AS4XX_AURIX_TC27X_CA_PB_MEM_V100
.\Tresos
```

2.3.2 Starting Up

There are three possible scenarios for your BSW Package which will be explained in detail.

2.3.2.1 MCAL Delivered Within Vector BSW Package

To figure out if this scenario is applied with your BSW Package, check whether the above-mentioned folders exist. If they do and contain the needed files (e.g. embedded source code, AUTOSAR description files etc.) no preparations are necessary.

2.3.2.2 MCAL Not Contained Within Vector BSW Package

With this scenario, you must do some preparations, e.g. install the MCAL etc.



Caution
In order to activate the MCAL and to use the workflows described later, some adaptions might be necessary on MCAL packages.

To ease this procedure, Vector provides a tool to execute these steps automatically. This tool also supports the user during MCAL package updates received from the MCAL manufacturer.



Note
Please consider that the different MCAL vendors may provide several packages. Sometimes it might be necessary to perform merge actions by hand before above-mentioned tool can be performed. For specific information please refer to the documents [4].



Practical Procedure
Please start the *3rd party MCAL Integration Helper* (*3rdPartyMcalIntegrationHelper.exe*) from the folder `ThirdParty\McalIntegrationHelper`.

For further information regarding this tool refer to chapter 2.5.1.



Expert Knowledge
For compatibility reasons the `Script_MCAL_Prepare.bat` is taken over, but usually does nothing else than calling above mentioned tool.

Using the script instead of the program allows additional actions via other parameters (described when starting the script on the command line).

The user may have the requirement to process all within an automated test suite, which results in a so called non-interactive mode of the GUI tooling. Non-interactive means that the tooling runs without GUI.

This requires that the GUI inputs are handed over via the command line interface.

For further information regarding the non-interactive mode refer to chapter 2.5.1.6.

The tooling processes the following principal steps (not all are necessary depending on the according to the specifications of the third party packages):

> Prepare MCAL configuration by copying of plug-ins or similar steps

> Rename MCAL provided specifics like `Compiler_Cfg.h` and `MemMap.h` within include paths (as those files are already contained within the Vector BSW Package)

> Copy (maybe rename of) BSWMD files from a location within the third party folder structure for one specific derivative to the location where the Vector tooling is expecting it. If needed you will be asked which derivative shall be taken.

> Adaptions on certain artifacts created and delivered by Vector to integrate the MCAL on several levels (e.g. Makefiles, Tooling-interfaces)

2.3.2.3 MCAL Update Needed

Usually within your development phase there might be updates of your third party MCAL available that have not been considered during delivery integration. Vector supports you for this use case and does some preparations within the MCAL Integration Package phase.

When you are receiving an update of the MCAL please clean up the MCAL Integration.

For this use case follow the notes in chapter 2.5.1.4.

2.4 Workflow

As mentioned in the introduction, there exist three reasonable workflows the user can choose from. Which one leads to the highest benefit-cost ratio depends on user requirements / preference on the one hand and on the level auf AUTOSAR conformance of the MCAL on the other hand. In the following all three workflows are introduced and their corresponding prerequisites and advantages/disadvantages are listed for guidance.

Note
Your BSW Package contains a document [4] which describes the recommended / supported workflow.

Multimedia Link
For the case that semiconductor supplies MCAL with EB tresos™ there is a video available which shows the MCAL integration as well as an exemplary workflow. We highly recommend having a look into this video for better understanding necessary inputs as well as interaction.

Just have a look at [5].

When you are not able to watch this video please get in contact with Vector delivery engineer to provide you an offline-version.

Note
Since the following chapters only mention the term DaVinci Configurator Classic some words upfront.

You will (have to) use DaVinci Configurator 4 for AUTOSAR3 based MICROSAR Classic product. With AUTOSAR4 based MICROSAR Classic product you will have to use DaVinci Configurator Classic.

For some special reasons, you might get DaVinci Configurator Classic even if your delivery is based on MICROSAR3 but this is not driven by topic MCAL, but on some others and therefore not explained in further details.

2.4.1 Single Configuration Tool Usage

After execution of the first steps described in 2.3.2 the user is able to use Vector DaVinci Configurator Classic as the exclusive configuration tool for both the MICROSAR Classic BSW as well as the third party MCAL. The configuration of the third party components is based on the MCAL AUTOSAR component definition files (BSWMD) which are (must be) included in the original semiconductor vendor packages.

You can activate/add the third party components to your configuration in the same way you would activate/add the MICROSAR Classic BSW components.

Vector has integrated the commands for the external code generators so that you will be able to start the MCAL code generation within the Configurator GUI by using this workflow. DaVinci Configurator Classic used to call the MCAL generator and displays warnings, errors, and other information.

Prerequisite	
AUTOSAR component definition files provided by MCAL manufacturer must be completely AUTOSAR conform and as consistent as possible to used proprietary files.	
Advantages	**Disadvantages**
Only one configuration tool and one configuration file to be used / no redundant data	No usage of convenience features which are possibly provided by MCAL manufacturer but not transferred/modeled within the AUTOSAR component definition files
All dependencies between BSW and MCAL components can be solved easily	

Table 2-1 Guidance for single configuration tool usage

2.4.2 Mixed Configuration Tool Usage

The configuration tooling respectively the internal (proprietary) description files of some third party MCAL contains internal logic which supports the user during configuration steps. Examples are

- Parameter A is only needed if Parameter B is set to value “X”
- Parameter C is calculated automatically based on A and B
- During component instantiation a list of containers is created automatically with reasonable and different values

These mechanisms are not at all or not completely specified by the AUTOSAR standard and not or not always modeled within the AUTOSAR component definition files provided. The fact is that they are not feasible outside the original MCAL configuration/generation tool. As a result and depending on the level of feature convenience of the MCAL configuration/generation tool, it is advisable to at least use it for the project setup as it will speed up the process in this phase.

By doing so, the user has to deal with two different tools during ECUC setup. But in a later phase, it will be easier to use a *one tool solution* as described in 2.4.1. Thus, the user will have to deal with a tool transition. To ease this procedure, Vector has introduced some tool features.

The diagram below shows a workflow to use both, the features of the MCAL tool and the convenience of a solution which is based on a single tool.

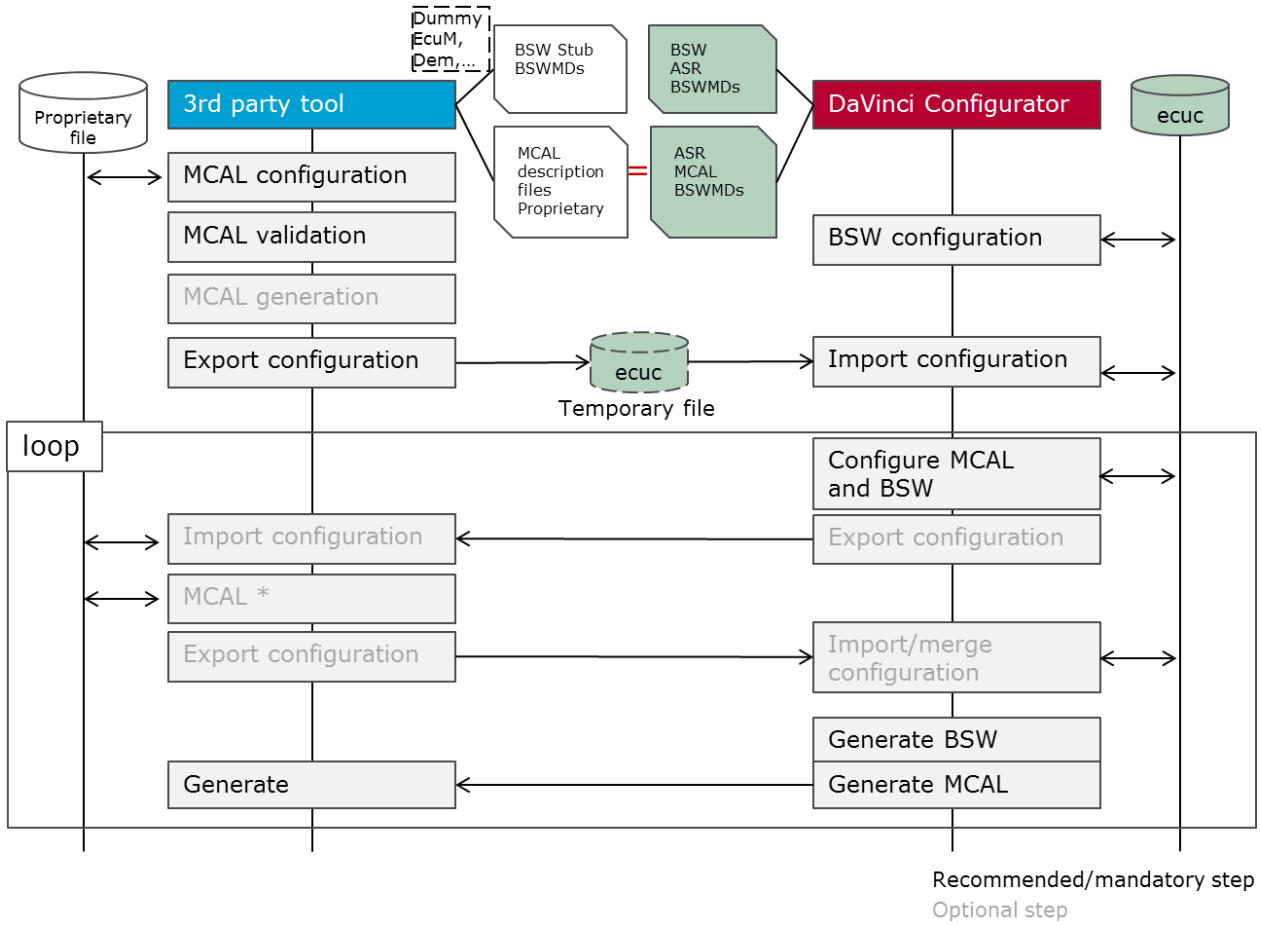


Figure 2-1 Configuration Workflow – Mixed configuration tool usage

Prerequisite	
AUTOSAR component definition files provided by MCAL manufacturer must be completely AUTOSAR conform and as consistent as possible to used proprietary files.	
Advantages	**Disadvantages**
Usage of convenience features of both tools	Additional tool knowledge is needed
Dependencies between BSW and MCAL components can be solved by data synchronization (export/import)	Overhead due to export/import workflow
Switch to single configuration tool is possible	Administration of two different tools is necessary

Table 2-2 Guidance for Mixed Configuration Tool Mode

Practical Procedure (start-up)

1. Create a DaVinci project
2. Activate BSW components needed, but no MCAL components
3. Create a project within the third party configuration tool (see 2.5.3)
4. Configure all MCAL components needed within the third party configuration tool
5. Export all MCAL components from the third party tool to an AUTOSAR conform ECUC file
6. Import this file in DaVinci Configurator Classic

Note

The third party tool might use a proprietary description format that deviates from the AUTOSAR BSWMD format. This might lead to inconsistencies between the formats and result in warnings and/or errors after import of the configuration in DaVinci Configurator Classic (e.g. due to different container multiplicities). Those errors might be handled via DaVinci Configurator Classic feature "Set User Defined" in the configuration GUI.

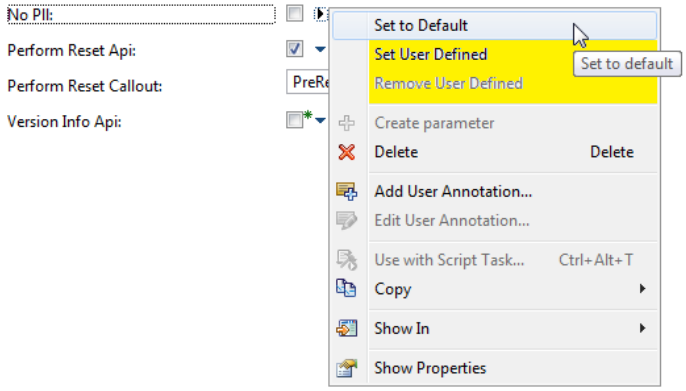


Figure 2-2 Da Vinci Configurator Classic: Set User Defined



Note
As marked in Figure 2-1 above, the loop - sometimes mentioned as round trip - is generally supported. Precondition for a successful round trip is DaVinci Configurator Classic 5.10 and above (this implies that with AUTOSAR3 based BSW and DaVinci Configurator 4 this scenario is not fully supported).

Round trip workflow has been evaluated with several semiconductor vendor's MCALs and is working in principle.

Based on the semiconductors AUTOSAR compatibility, there may raise some warnings or sometimes even single parameters may be reconfigured, based on insufficient importer functionality.

2.4.3 Split Configuration Tool Usage

This workflow is required if the third party configuration tool is not completely AUTOSAR compliant as it, e.g., violates the AUTOSAR standard parameter definition or does not support a lossless ECUC roundtrip with DaVinci Configurator Classic or other AUTOSAR tools.

As the number of interfaces respectively the number of exchanged parameters between the AUTOSAR BSW and the MCAL components is comparatively small, it may also be a solution to use both tools permanently in parallel. The implication is that the code generation will be triggered from the tool in which the configuration has been made.

To integrate both parts on embedded side, there is still the need to share some parameters between both configuration tools. This can either be achieved by manual replication (using configuration stubs, e.g., DEM stub of third party MCAL, MCU stub of Vector) or by using the import export mechanisms introduced in 2.4.2.

Prerequisite	
AUTOSAR component definition files provided by MCAL manufacturer must be available but must be matching to proprietary formats only partly (at interfaces between BSW and MCAL).	
Advantages	**Disadvantages**
Relatively tolerant workflow in case that AUTOSAR component definition files and/or third party tools are not completely AUTOSAR conform	Dependencies between BSW and MCAL components must be solved manually
	Additional tool knowledge is needed
	Overhead due to 2 tool workflow and redundant configuration data → manual synchronization needed

Table 2-3 Guidance for Split Configuration Tool Usage

2.5 Configuration Tools

In this chapter, some notes for the configuration tools included in your package are given.



Note
Most common tooling used by third party MCAL supplier is EB tresos™, therefore the usage is explained extensively.

2.5.1 3rd Party MCAL Integration Helper Tool

The 3rd Party MCAL Integration Helper tool handles the use cases which come up with the third party packages from third party MCAL supplier which you want to use in your project.

The most frequently used use cases are presented in the following chapters:

- First Integration of third party Packages
- Change of Derivative
- Add additional third party Packages
- Update of third party Packages
- Remove all third party Packages

The user may have the requirement to process all within an automated test suite, which results in a so called non-interactive mode of the GUI tooling. Non-interactive means that the tooling runs without GUI. This requires that the GUI inputs are handed over via the command line interface. For further information regarding the non-interactive mode refer to chapter 2.5.1.6.



Caution
Never copy, delete and modify manually the third party packages in your Vector MCAL Integration Package. Please use the 3rd Party MCAL Integration Helper tool instead. The standard path where the third party packages are installed is the Supply folder. This folder is under control of the tool and it is the starting point for integration activities of third party packages into your project.

2.5.1.1 First Integration of third party Packages

Usually, your Vector MCAL Integration Package does not contain specific third party packages. Hence if you have selected the third party packages you want to use in your project, this tool integrates them into it.

Practical Procedure - Overview

> Install the third party MCAL on your local computer
> Check the installed MCAL for completeness
> Check the installed MCAL for correct derivative support
> Run the 3rd Party MCAL Integration Helper tool
> Proceed with other tools like EB tresos™ and/or DaVinci Configurator Classic

After installation of the desired third party MCAL the 3rd Party MCAL Integration Helper tool will support you in integrating this MCAL into the current project.

Note
Do not rename the third party packages from third party vendor which you want to use in your project. Otherwise the 3rd Party MCAL Integration Helper tool maybe does not recognize if the package is a valid third party package for your project.

Start the tool by executing 3rdPartyMcalIntegrationHelper.exe.

The tool recognizes that currently no packages are available in the Supply folder of your project and proposes to copy the mandatory packages into it. The mandatory packages are the ones which are at least necessary for a successful integration:

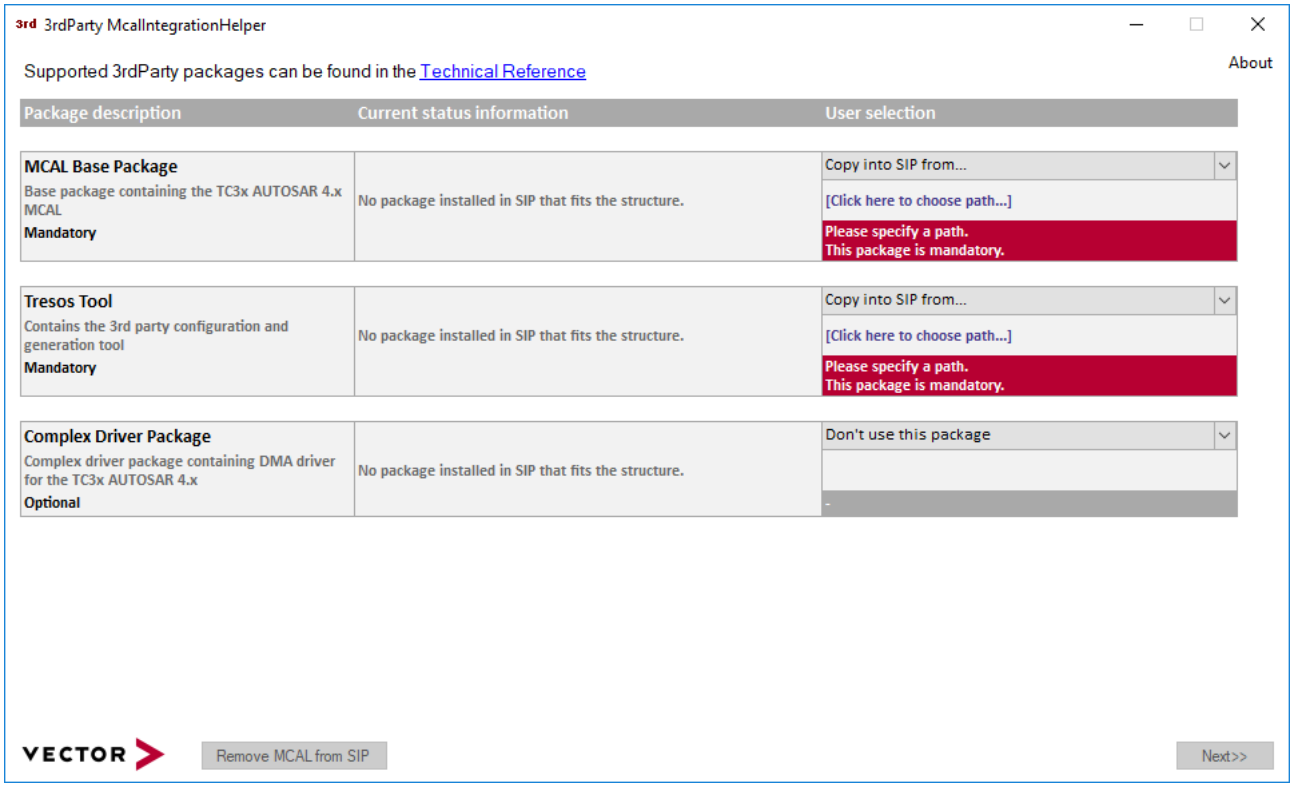


Figure 2-3 3rdPartyMcalIntegrationHelper: First Integration

Specify the path to the current third party packages like the MCAL and EB tresos™ package and click on the “Next” button.



Note
3rd party MCAL Integration Helper tool does not support the specification of package paths with UNC path.

The tool will copy the third party packages into the standard path in Vector MCAL Integration Package. This is the folder `Supply`. After that it will proceed in integrating the third party packages into Vector MCAL Integration package. Within this process you might be asked to select a derivative supported by the MCAL.

Any kind of problems will be reported in the log file 3rdPartyMcalIntegrationHelper.log which is next to the 3rdPartyMcalIntegrationHelper.exe.

After the successful integration of the third party packages, you can proceed with other tools to modify and generate your project such as the DaVinci Configurator Classic and external generation tool, e.g. EB tresos™.

2.5.1.2 Change of Derivative

Possibly in the first integration of the third party packages into your project, you were asked to select a derivative supported by your MCAL. If you want to change this selection, the 3rd Party MCAL Integration Helper needs to update your project.



Practical Procedure - Overview
> Run the 3rd Party MCAL Integration Helper tool
> Keep all packages in tool and step to derivative selection
> Let the tool rerun the integration
> Update and generate the project configuration

Start the tool by executing 3rdPartyMcalIntegrationHelper.exe.

The tool recognizes the current packages and proposes to keep them:

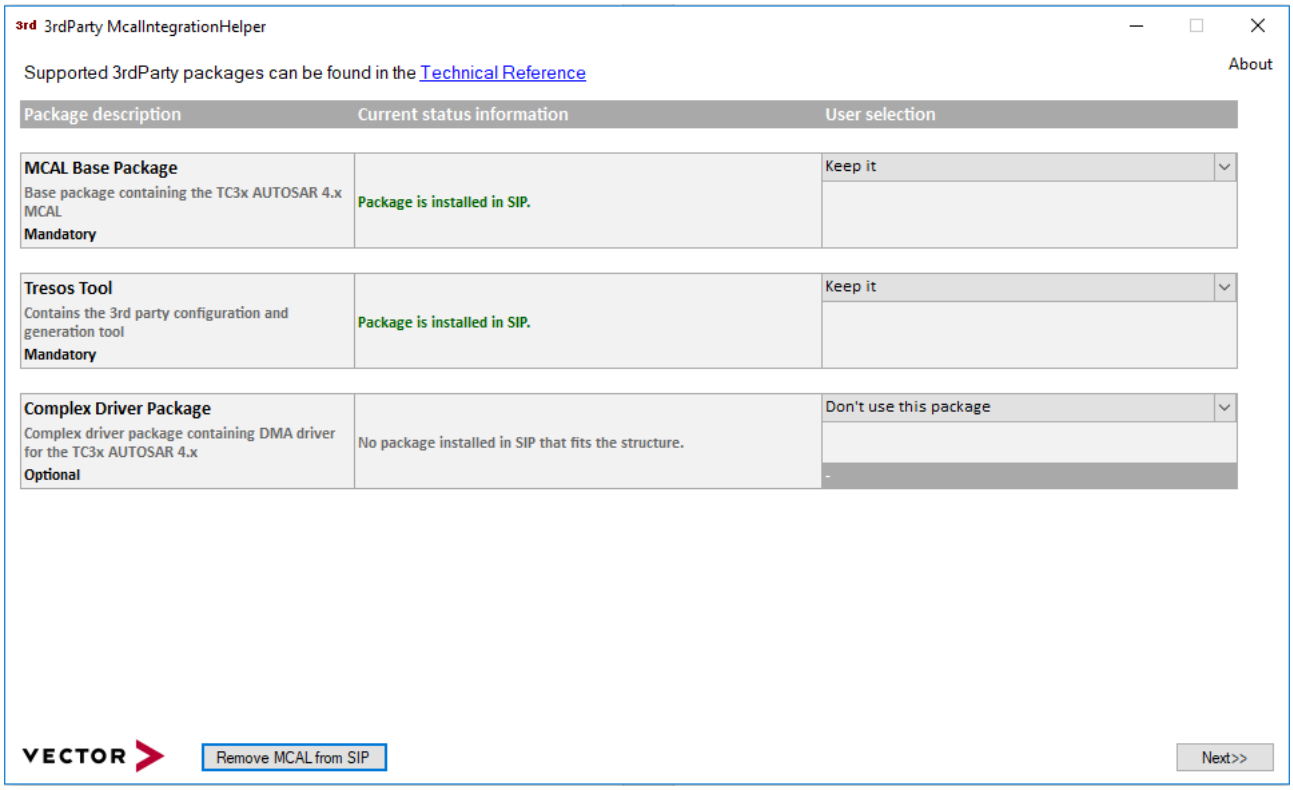


Figure 2-4 3rdPartyMcalIntegrationHelper: Keep Packages

Click the “Next” button: the tool will start the integration after you click the “Start” button.

Click the “Next” button when tool has finished: next step will offer all available derivatives:

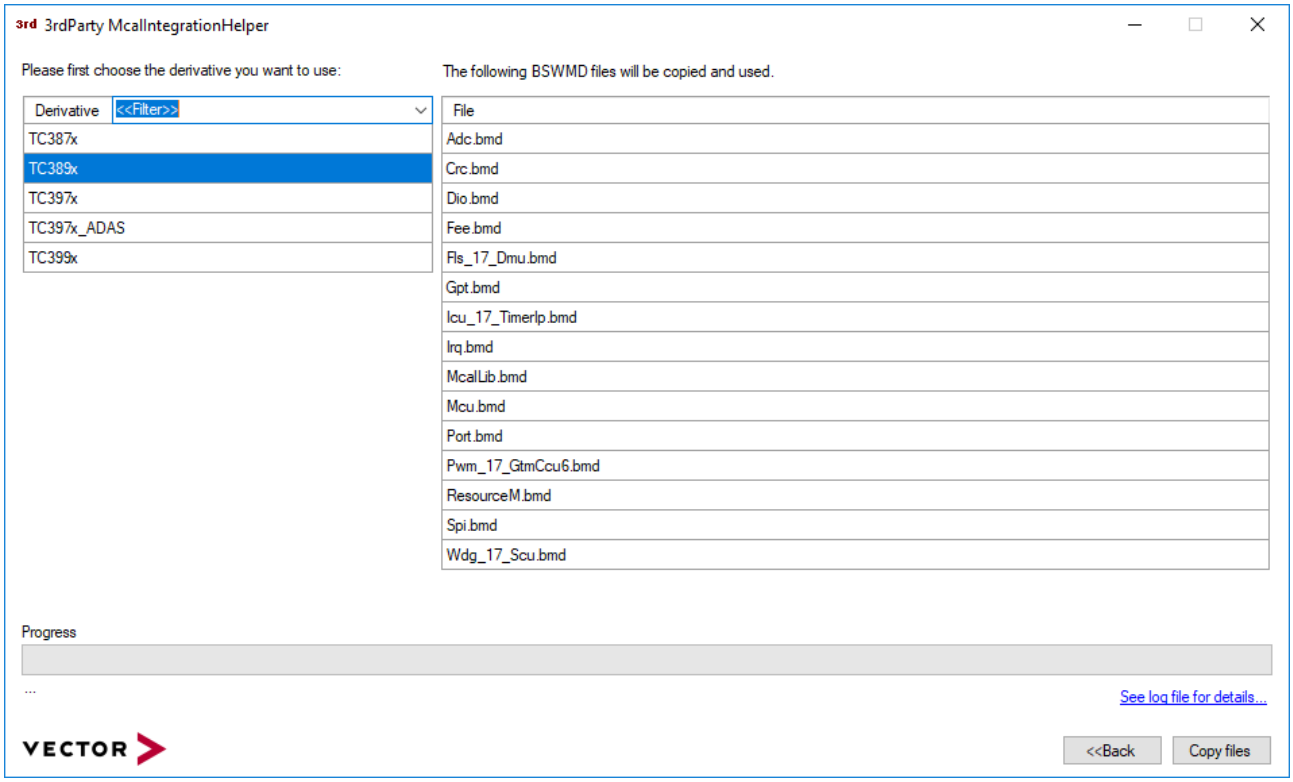


Figure 2-5 3rdPartyMcalIntegrationHelper: Derivative Selection

After selecting a derivative, the tool shows all BSWMD files associated with chosen derivative. By clicking “Copy files” button these files will be integrated into your project.

Let the tool run until “Exit” button is shown. Click that button and the tool has finished the update of your project with the new derivative.

Any kind of problems will be reported in the log file 3rdPartyMcalIntegrationHelper.log which is next to the 3rdPartyMcalIntegrationHelper.exe.

Update and generate the project configuration with DaVinci Configurator Classic and external generation tool, e.g. EB tresos™.

2.5.1.3 Add additional third party Packages

Additional packages are e.g. Memory packages or Safety relevant packages which belong to an MCAL Base package. These packages can be integrated into your project any time.



Caution
Vector cannot ensure full compatibility between additional and MCAL Base package because the compatibility depends on the third party vendor´s specification. Please take care of this issue and check if packages fit to each other.



Practical Procedure - Overview

> Install the additional third party packages on your local computer
> Check the installed packages for completeness
> Check if the installed packages fit to your MCAL Base package e.g. for correct derivative support
> Run the 3rd Party MCAL Integration Helper tool
> Update and generate the project configuration

After installation of the desired additional third party packages, the 3rd Party MCAL Integration Helper tool will support you in integrating them into the current project.



Note
Do not rename the third party packages from third party vendor which you want to use in your project. Otherwise, the 3rd Party MCAL Integration Helper tool maybe does not recognize if the package is a valid third party package for your project.

Execute the 3rdPartyMcalIntegrationHelper.exe.

The tool shows the current packages in the folder `Supply` and additional packages which are currently not in this folder. The latter are set to “Don´t use this package”:

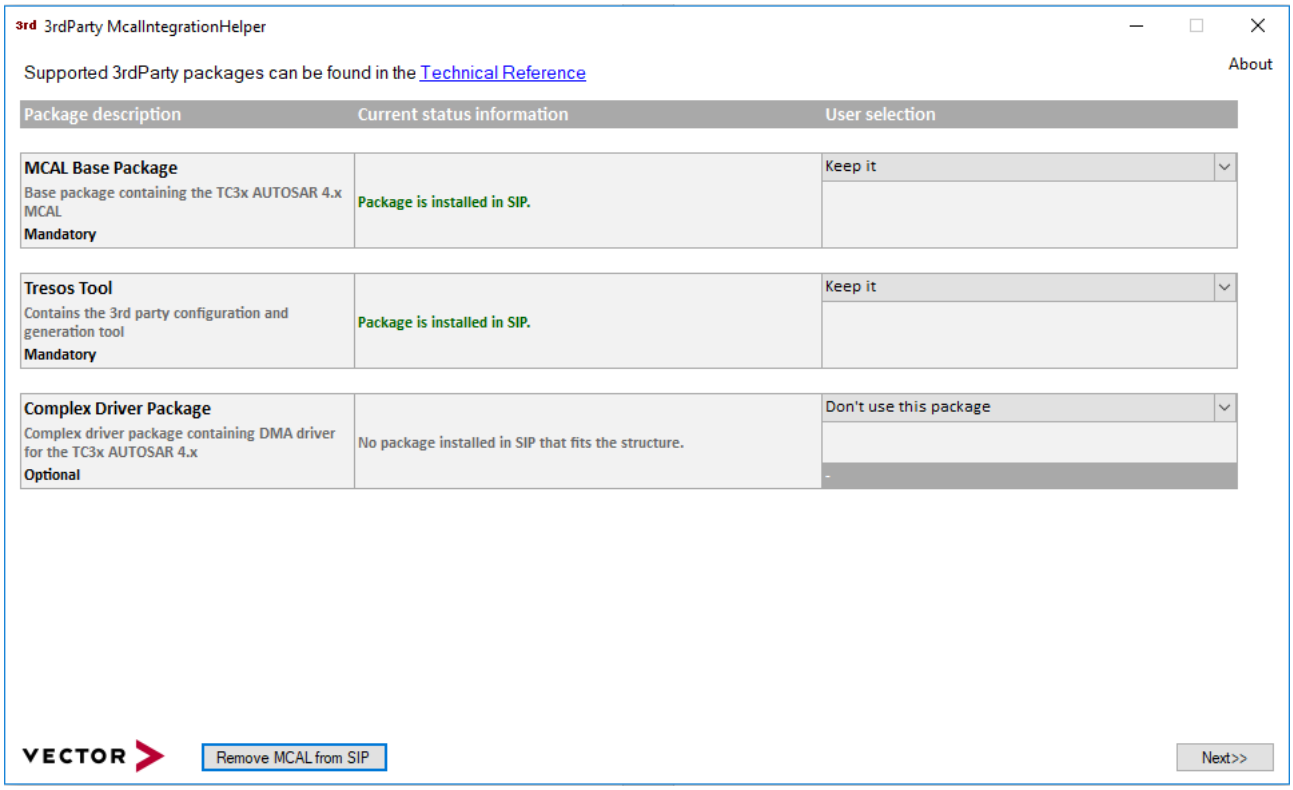


Figure 2-6 3rdPartyMcalIntegrationHelper: Additional Packages

Change the user selection from “Don´t use this package” to “Copy into BSW Package from….” and enter the path to your additional package which fits to your MCAL Base package.

Click on “Next” button and let the tool run until “Exit” button is shown. Click that button and the tool has finished the integration of the additional third party packages into your project.

Any kind of problems will be reported in the log file 3rdPartyMcalIntegrationHelper.log which is next to the 3rdPartyMcalIntegrationHelper.exe.

Update and generate the project configuration with DaVinci Configurator Classic and external generation tool, e.g. EB tresos™.

2.5.1.4 Update of third party Packages

This chapter supports you in updating the third party MCAL (e.g. due to a new version) in the current project. This description is an example of how this could be done and what the challenges are. The workflow might depend on the third party MCAL itself and its changes.

These are the benefits to the customer when using the update feature:

- No BSW Package update needed (avoids extra costs and efforts)
- The project can be updated within the existing BSW Package
- The necessary update of the MCAL Integration Package is done with the help of the 3rdPartyMcalIntegrationHelper tool

> If the new MCAL has additional memory sections and Exclusive Areas, these can be processed using an additional tool called 3rdPartyMcalMemMap tool



Caution
Vector cannot ensure full functionality of the update process because the compatibility to the given MCAL Integration Package deeply depends on the changes within the new third party MCAL.

Such changes like structural differences, additional memory sections and Exclusive Areas might cause compiler / linker errors or even runtime issues. The additional memory sections and Exclusive areas can be detected using the 3rdPartyMcalMemMap tool. Please refer to the document [8] for more information on using this tool.



Caution
Since the update process is quite supplier specific please refer to the document [4] provided with the BSW Package for further details.



Practical Procedure - Overview

> Install the third party MCAL on your local computer
> Check the installed MCAL for completeness
> Check the installed MCAL for correct derivative support
> Run the 3rd Party MCAL Integration Helper tool
> Start the 3rdPartyMcalMemMap.exe to find additional memory sections and Exclusive Areas
> Update and generate the project configuration



Note
If you update the third party MCAL packages in your project, you must also update the additional packages or at least check if the versions of MCAL Base package and additional packages fit to each other.

After installation of the desired third party MCAL the 3rd Party MCAL Integration Helper tool will support you in integrating this MCAL into the current project.

Start the tool by executing 3rdPartyMcalIntegrationHelper.exe.

The tool recognizes the current packages and proposes to keep them:

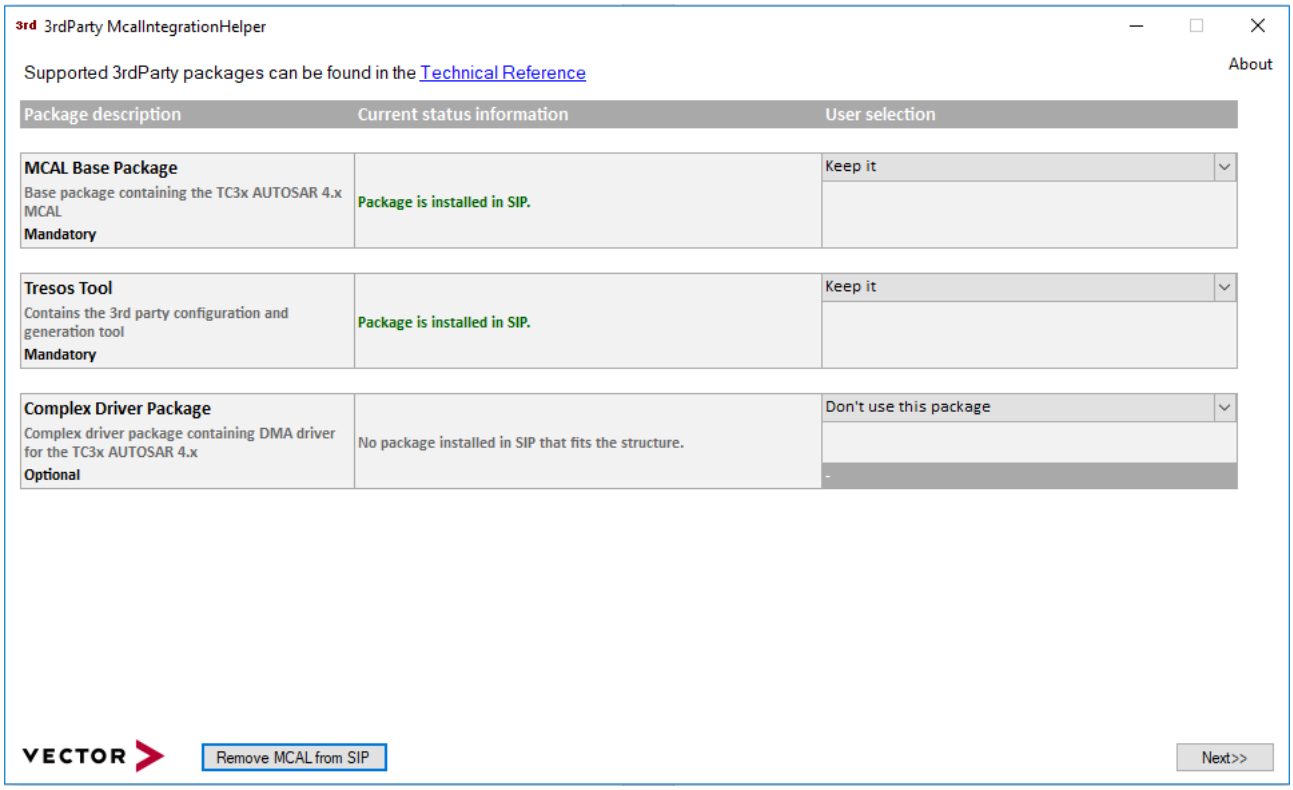


Figure 2-7 3rdPartyMcalIntegrationHelper

Select “Replace it with…” instead of “Keep it” for the packages you want to update and enter the path to the new packages:

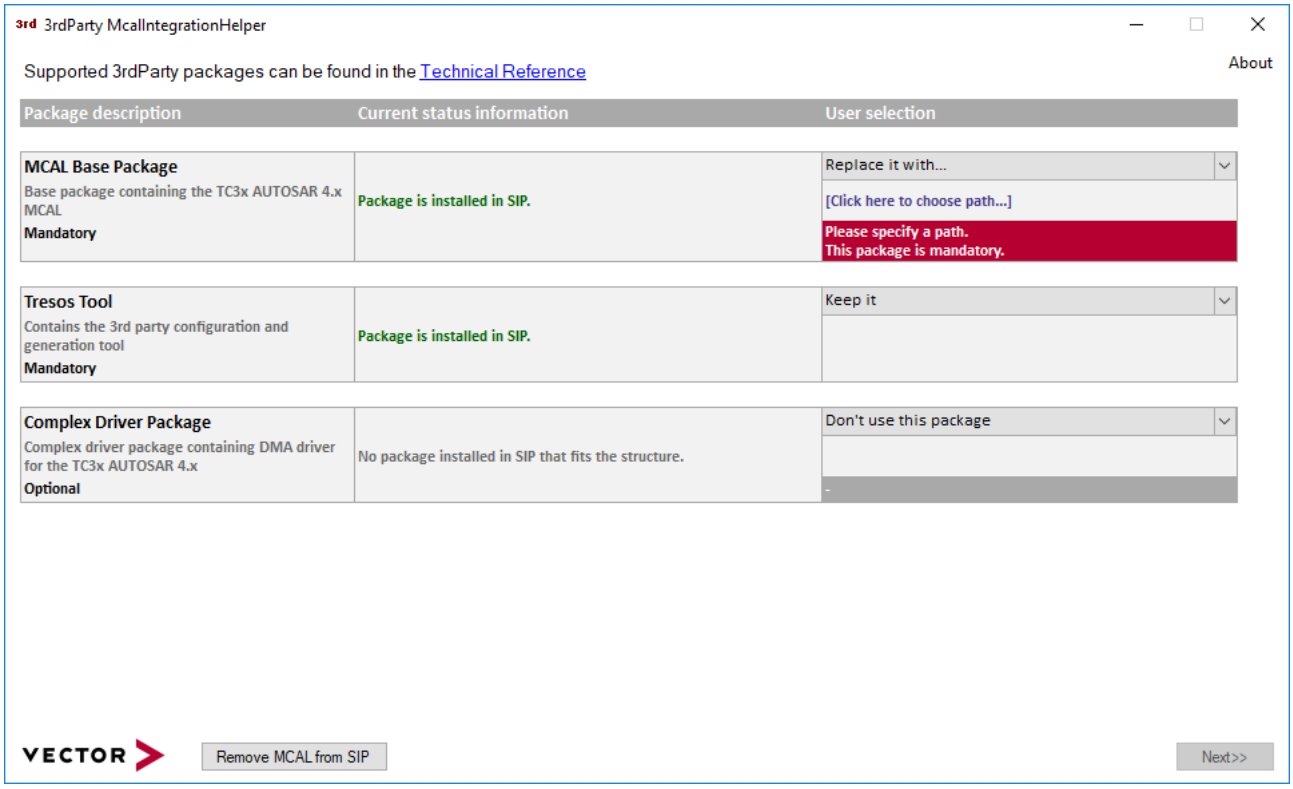


Figure 2-8 3rdPartyMcalIntegrationHelper Package Update

Click on “Next” button and the tool will proceed in updating the third party packages and the Vector MCAL Integration Package.

Any kind of problems will be reported in the log file 3rdPartyMcalIntegrationHelper.log which is next to the 3rdPartyMcalIntegrationHelper.exe.

Update and generate the project configuration with DaVinci Configurator Classic and external generation tool, e.g. EB tresos™.

Note

After MCAL update, when opening your project in DaVinci Configurator Classic, you might be informed that files have changed. This will only happen in case that the AR-PACKAGE name of a module changed. In the next window click on “<Ignore>” and select the new “Modul Definition” for each MCAL module.

The selection of the new definition files is shown in following screenshot of DaVinci Configurator Classic:

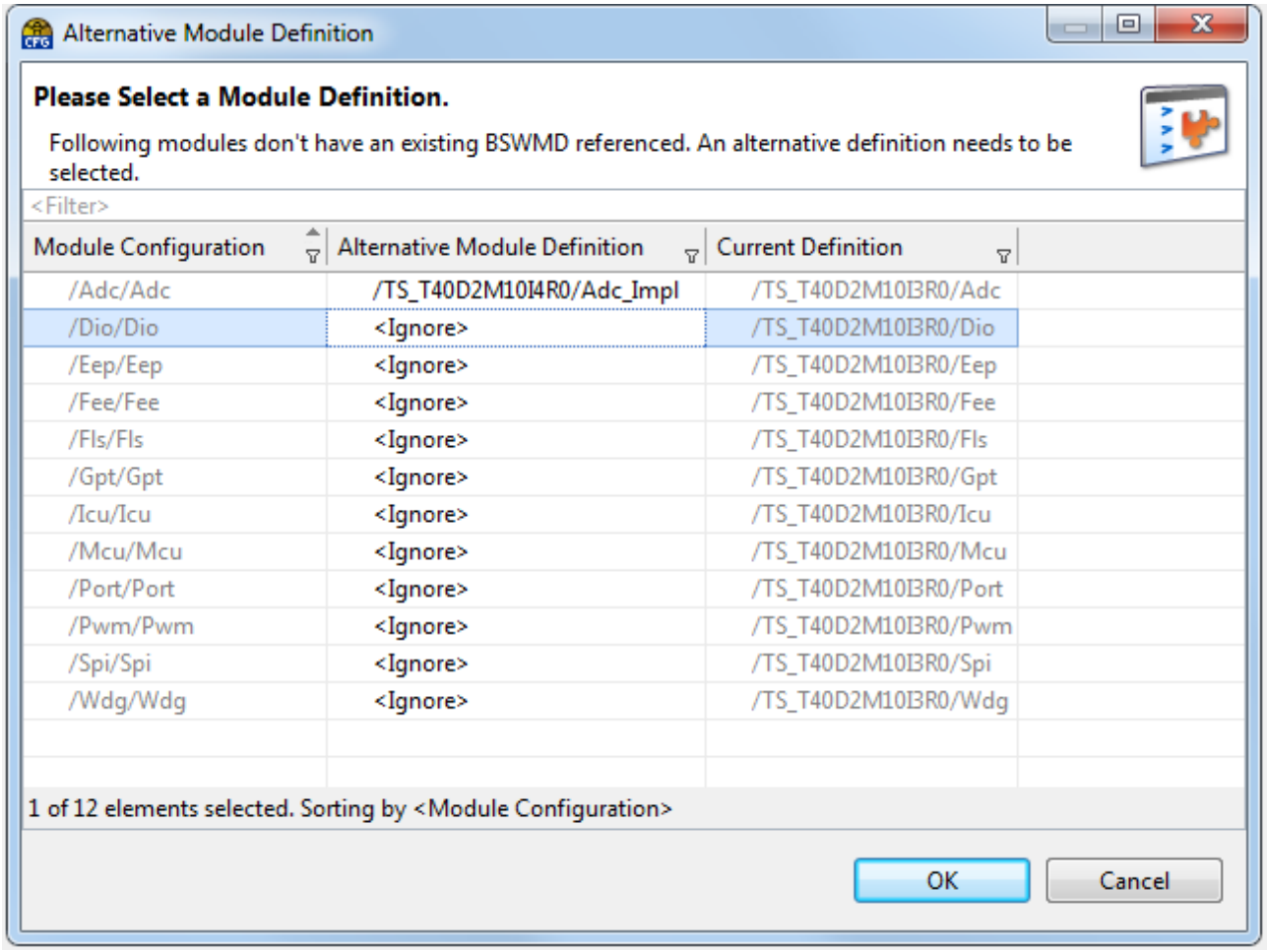


Figure 2-9 DaVinci Configurator Classic: Updating Modul Definitions

During configuration you might recognize some errors in the GUI. DaVinci Configurator Classic can support you by selecting the solving actions (e.g. “Solve with default Action”).

Some other manual changes on your configuration may be necessary due to the MCAL update.

2.5.1.5 Remove all third party Packages

The 3rd Party MCAL Integration Helper tool offers the possibility to remove the third party packages and to revert the changes done by previous integration from your project.

Execute therefore the 3rdPartyMcalIntegrationHelper.exe.

The tool recognizes the current packages in Supply folder and activates the button “Remove MCAL from BSW Package” at the bottom of the window:

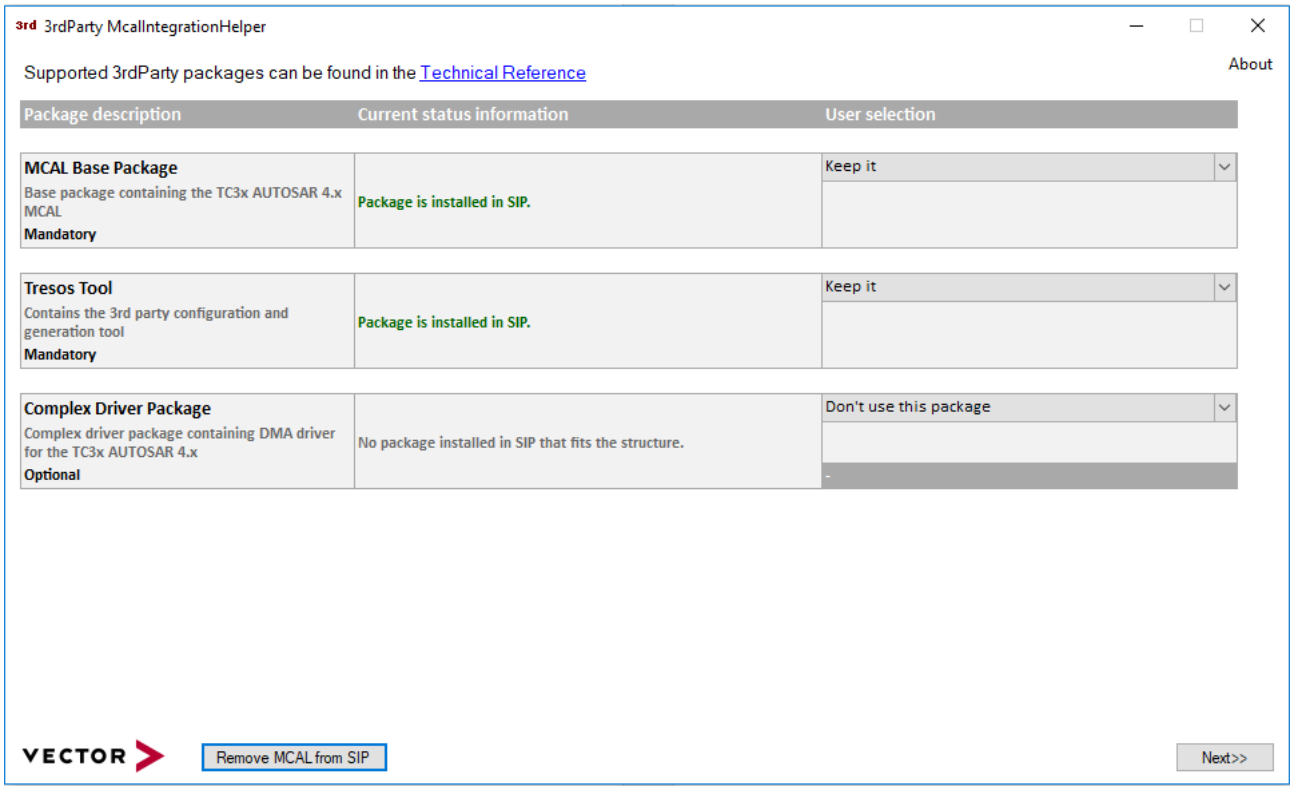


Figure 2-10 3rdPartyMcalIntegrationHelper: Remove Packages

Click the button “Remove MCAL from BSW Package” and let the tool run until “Exit” button is shown. Click that button and the tool has finished the deletion of the third party packages from Supply folder and the revert of the integration from your project.

Any kind of problems will be reported in the log file 3rdPartyMcalIntegrationHelper.log which is next to the 3rdPartyMcalIntegrationHelper.exe.

Update and generate the project configuration.

2.5.1.6 Using the Non-Interactive Mode

The 3rd Party MCAL Integration Helper tool offers the possibility to be executed in a so called non-interactive mode which means the tooling runs without GUI. Use the `Script_MCAL_Prepare.bat` located in the folder `ThirdParty\<MCAL_µC>\VectorIntegration` to start the tool via command line. This script provides several options:

Command	Description
`--help`	Displays an overview of the available options
`--prepare`	Opens the 3rd Party MCAL Integration Helper tool GUI
`--auto DERIVATIVE`	Prepares the third party packages in the Supply folder for usage
`--undo`	Inverts the steps from option “auto”
`--copy ID PATH DERIVATIVE`	Copies one third party package in the Supply folder and prepares it for usage
`--replace ID PATH DERIVATIVE`	Replaces one third party package from the Supply folder with another third party package and prepares it for usage
`--ignore ID DERIVATIVE`	Ignores one third party package from the Supply folder during preparation of the other packages

Table 2-4 Command Line Options of 3rd Party MCAL Integration Helper Tool

The above options may require the specification of the parameters `ID`, `PATH` and `DERIVATIVE`, which are described in the following subchapters.

2.5.1.6.1 DERIVATIVE

This parameter specifies the derivative identifier given by the user. This is similar to the derivative selection step in the GUI mode.



Expert Knowledge
The derivative name displayed in GUI mode may be different from the one specified as parameter for the non-interactive mode. Detailed information can be found in the file “MIPconfig.xml” within the tag `<DerivativeScan/>` or `<Derivatives/>`. The required information can be found with the name `IdentifyingFileFragment`.

If the use of regular expressions is not practiced, the steps of the Practical Procedure below should be followed instead to find the correct derivative identifier:



Practical Procedure - Overview
> Run the 3rd Party MCAL Integration Helper tool in GUI mode

> Open the log file 3rdPartyMcalIntegrationHelper.log
> Go to section “Determine available derivatives”
> Search desired derivative in the list of found derivatives
> Copy the value of the `IdentifyingFileFragment`
> Use the value as DERIVATIVE identifier at the correct position in the command line

The section of the log file mentioned above could look like this:

```
================================================================================
Determine available derivatives
================================================================================
Found derivative                    |  Name=s32k118 - lqfp48, IdentifyingFileFragment=s32k118_lqfp48
Found derivative                    |  Name=s32k118 - lqfp64, IdentifyingFileFragment=s32k118_lqfp64
Found derivative                    |  Name=s32k142 - lqfp100, IdentifyingFileFragment=s32k142_lqfp100
Found derivative                    |  Name=s32k142 - lqfp48, IdentifyingFileFragment=s32k142_lqfp48
Found derivative                    |  Name=s32k142 - lqfp64, IdentifyingFileFragment=s32k142_lqfp64
Found derivative                    |  Name=s32k144 - lqfp100, IdentifyingFileFragment=s32k144_lqfp100
```

Figure 2-2-11 3rdPartyMcalIntegrationHelper.log: Available Derivatives

2.5.1.6.2 ID

This parameter specifies the id of the third party package that should be handled. This is necessary because in non-interactive mode only one package can be handled at a time and this package is identified by its id.

Note

The Id can be found in the file "MIPconfig.xml" within the `<Package/>` tag of the corresponding third party package. More specifically the value of the attribute `Id` is used.

An example of the section in the MIPconfig.xml file mentioned above could look like this:

```
<!-- Package 1 -->
<Package Summary="MCAL Package" Id="1">
    <NameExpected>(?!.*CSEC.*)S32K1.X.*MCAL.*4.*</NameExpected>
```

Figure 2-2-12 MIPconfig.xml: Package Id

2.5.1.6.3 PATH

For the options `copy` and `replace` an additional parameter `PATH` is necessary. This parameter specifies the path to the third party package that should be copied or used for the replace. The specified path has to lead to a compatible third party package.

A non-interactive mode call of the 3rd Party MCAL Integration Helper tool via command line, which replaces a specific third party package, can look like this:

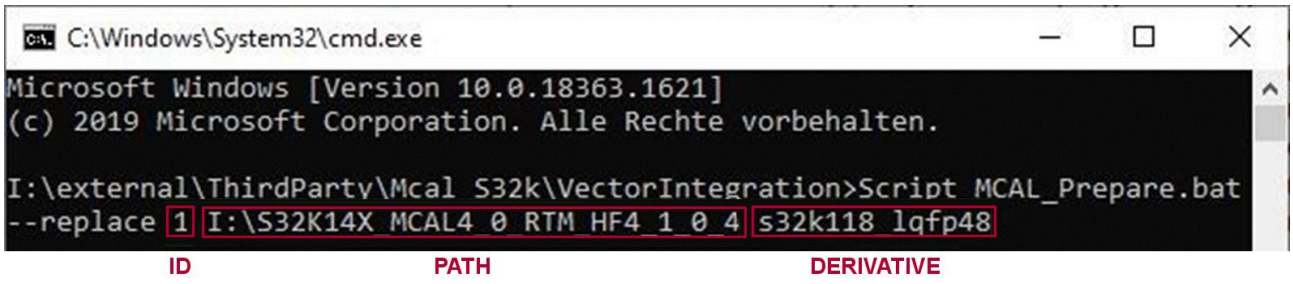


Figure 2-2-13 Command Line: Replace Call

2.5.2 Vector DaVinci Configurator Classic

Use DaVinci Configurator Classic to configure your MICROSAR Classic BSW using a DPA project. The MSR3 approach using DaVinci Configurator 4 or any previous configuration tool is not considered.
Please refer to the start-up manual included in the delivery. In addition, the DaVinci Configurator Classic provides user support via the help menu.

2.5.3 EB tresos™

Reference
Additional and useful information can be found here [5].
Regarding questions about tooling license, additional information can be found there [3].
Such documents are only content of affected BSW Packages.

Note
EB tresos™ is a tool developed by Elektrobit company. In the Vector environment it might be used as external configuration and generation tool for several third party MCAL packages from different vendors and enables the user creating the dynamic MCAL source code.

2.5.3.1 EB tresos™ Classic

The program itself is located in the folder beneath: `ThirdParty\<MCAL_µC>\Supply\`

To start the tool the user must execute `tresos_gui.exe` which can be found usually beneath `Tresos\bin`. Just to note, some third party MCAL suppliers also provide 64bit versions of EB tresos™.

A useful documentation when you are struggling with a task can be found within the EB tresos™ folder structure by searching for a document named "*Studio_documentation_users_guide*" beneath `Tresos\doc` or other related documentation.

Note
As EB tresos™ is an external tool not provided from Vector and several versions exist used with different MCAL vendors, MCAL platforms and MCAL versions, we cannot ensure that the following pictures from the tool layout equal the tool layout within your project.

2.5.3.1.1 Setting up a New Configuration Project

> To create a new project the Project Wizard must be started by to selecting File → New → Configuration Project.

> The project name as well as the workspace location plus the relevant AUTOSAR Release Version can now be chosen.

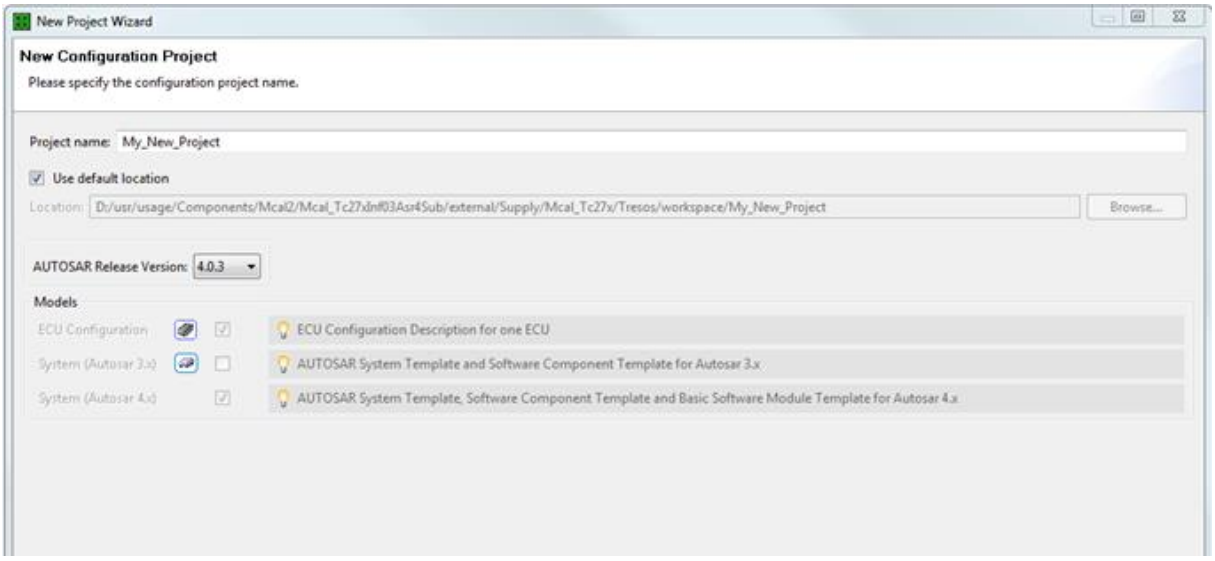


Figure 2-14 EB tresos™ User Interface: New Configuration Project

Note
The workspace location can be chosen freely and the path for code generation output can be set in the next step

2.5.3.1.2 Project Details

In the next step, the wizard requests the following data:

> ECU ID: fill in a symbolic name

> Target: Select your microcontroller target

> Generation Path: please choose the settings according to the settings within the DaVinci Configurator Classic to keep consistency and avoid problems regarding make process

> The remaining choices can be selected such as in the figure below

> We recommend activating the check box to ‘Automatically add the minimum number of child elements in lists’

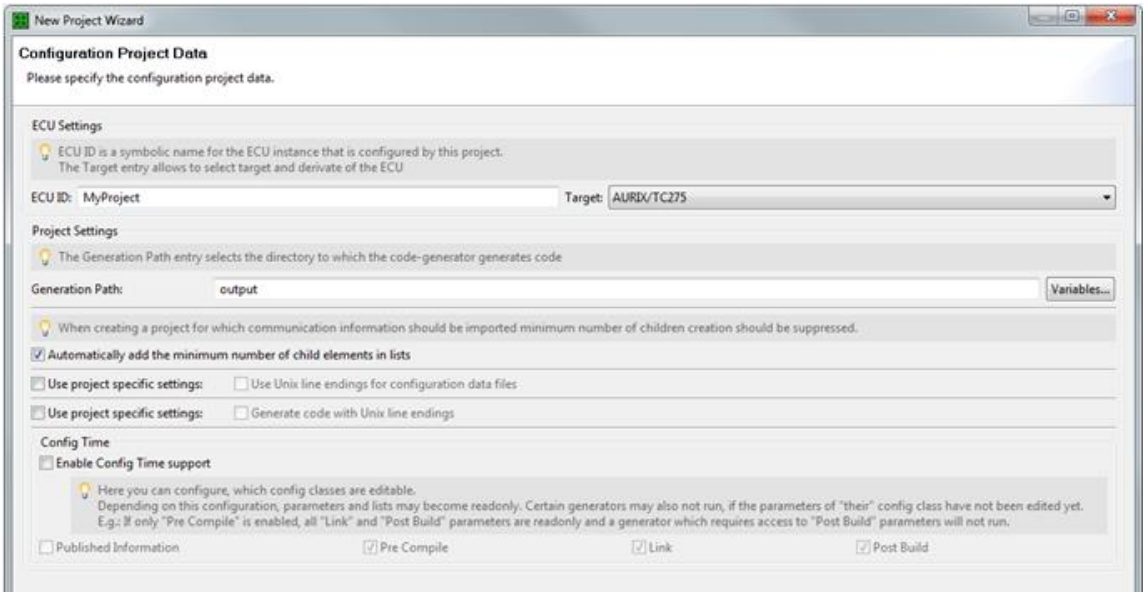


Figure 2-15 EB tresos™ User Interface: Configuration Project Data

Note
Several project settings can be changed after creation of the project by editing the file `preferences.xdm` located within the created workspace folder; e.g. adapt the generation path by changing the default value ‘output’ to a relative or absolute path (`<d:var name=”GenerationPath” value=”output”/>`).

2.5.3.1.3 Selection of Components

Now choose the components, which should be added to the project. In general, every MCAL component should be selected. In that case, use STRG + A to select all and click on the tagged button.

Attention: Some additional “dummy” components (stubs) might be necessary in order to configure the MCAL components e.g. Base, Resource, Dem or EcuM. Those components are only used to provide parameters and references needed by the MCAL components such as *Dem Events* and *EcuM run modes*, please refer to 2.5.4.

The window should now be similar to this:

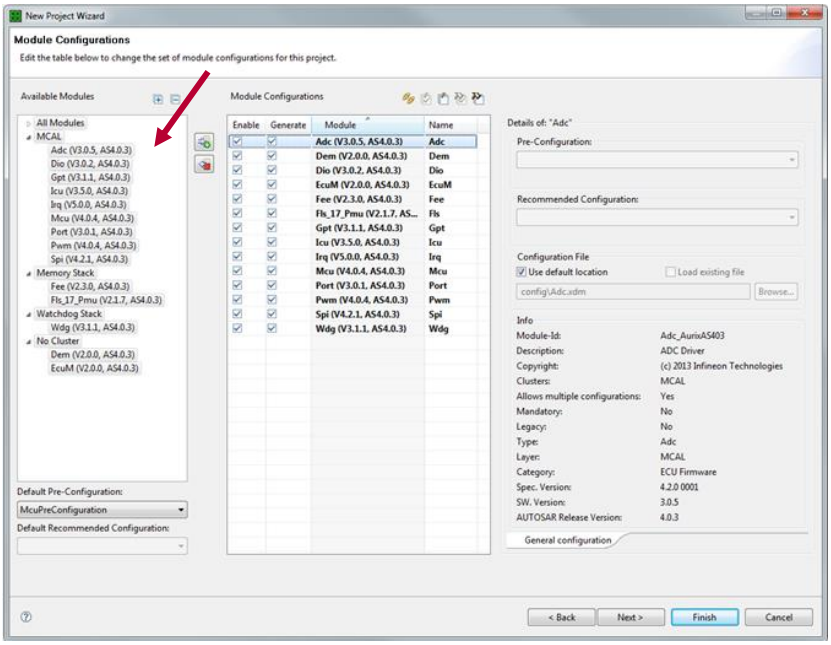
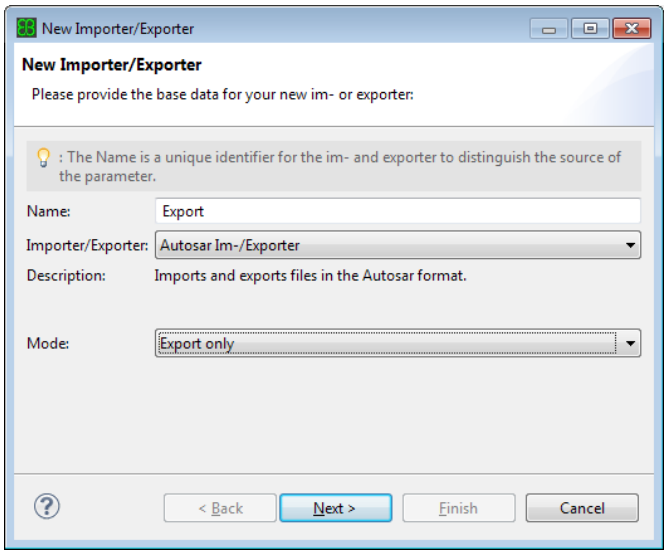


Figure 2-16 EB tresos™ User Interface: Component Configurations

2.5.3.1.4 Creation of Importers and Exporters

Now the user has the possibility to create importer and/or exporter to create a configuration file in an AUTOSAR standardized format out of the proprietary EB tresos™ configuration files. This functionality is needed to enable the workflows described in chapter 2.4.2 and 2.4.3.

Please Click on + to add an Im-/Exporter and use the following settings:



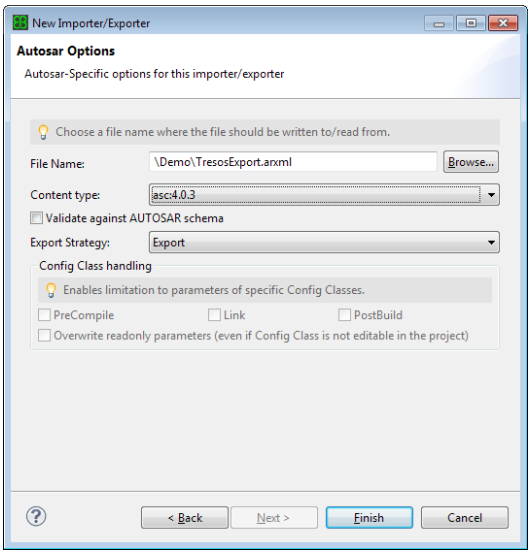


Figure 2-17 EB tresos™ User Interface: Create an exporter (step 1)
Figure 2-18 EB tresos™ User Interface: Create an exporter (step 2)

The importer can be configured in a similar manner.

If you don’t create the im-/exporters during setup you can do this later by using the context menu on the corresponding project.

2.5.3.1.5 Configure the MCAL Components

Now the wizard is finished and the MCAL itself can be configured.

2.5.3.1.6 Generation of the third party MCAL

If everything is configured, click on the generation button. EB tresos™ will now generate your project (note: there will be a validation phase triggered before to ensure that configuration is formal correct).



Figure 2-19 EB tresos™ User Interface: Generate Button

Note
The generation result from EB tresos™ is only needed for the third workflow (see chapter 2.4.3), but it might be recommendable to compare the generation results between the configuration tools to find inconsistencies between the different definition formats (EB tresos™ XDM versus AUTOSAR BSWMD) also with another workflow.

2.5.3.2 EB tresos™ Next Generation

The program itself is located in the folder beneath: `ThirdParty\<MCAL_µC>\Supply\`

To start the tool the user must execute `studio.exe`.

Since not all configuration parameters of EB tresos™ can be set on the CLI, the creation of a tresos project is mandatory.

2.5.3.2.1 Setting up a New Configuration Project

> To create a new project the Project Wizard must be started by to selecting File → New → Project and configuration projected must be chosen on the next screen

> The project name as well as the workspace location plus the relevant AUTOSAR Release Version can now be chosen.

> Finish the new configuration project.

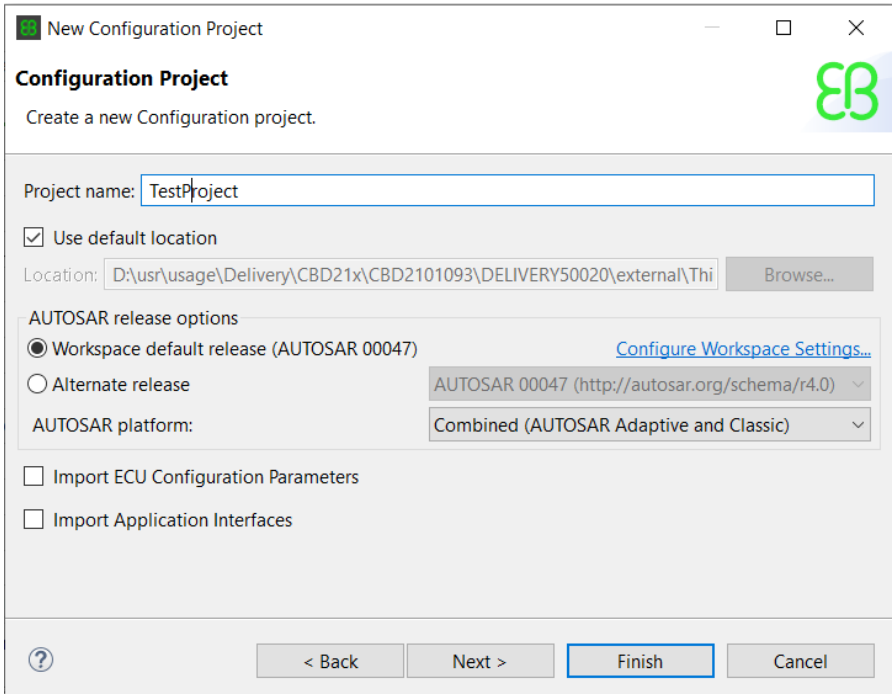


Figure 2-20 EB tresos™ User Interface: New Configuration Project

2.5.3.2.2 Project Details

To set the project details, clock right on the new project and choose properties:

> Target: Set target can be set in “EB AUTOSAR Tooling”

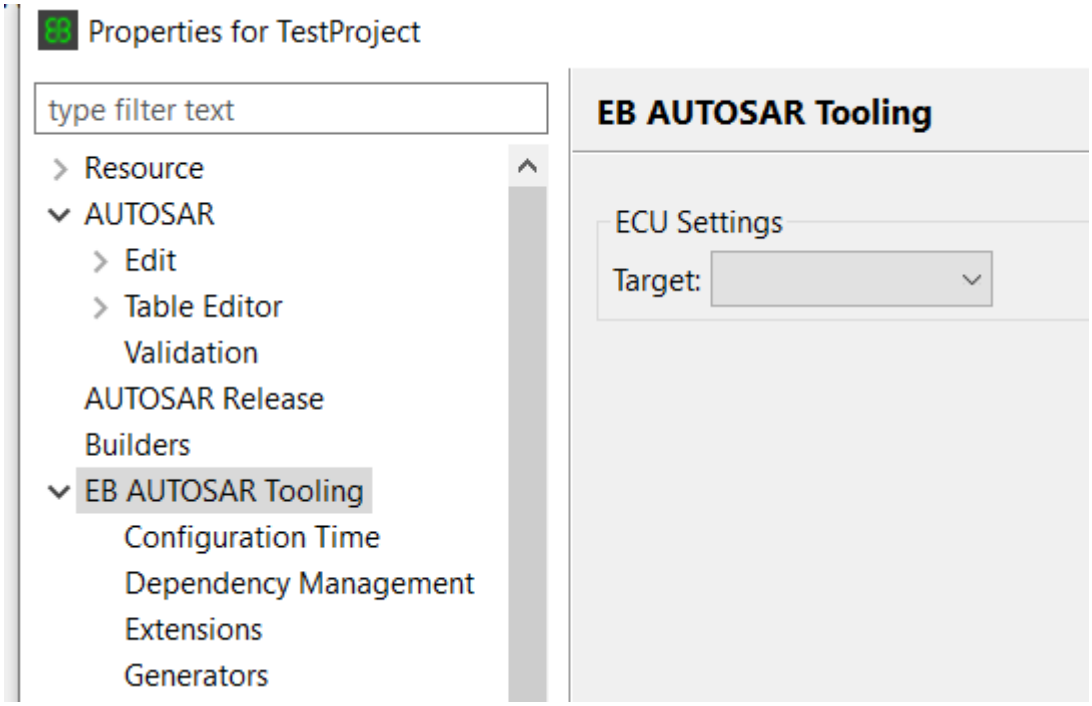


Figure 2-21 EB tresos™ User Interface: Set Target of the Project

> Extensions: in extensions, the desired resources and modules must be enabled:

- EcuResource: only the derivative which shall be used may be enabled
- Generators: All plugins which shall be generated must be enabled. Stub plugins like Dem, EcuC, EcuM and Os shall not be enabled
- Modules: all used modules including stub plugins shall be enabled here.

- Generator: the default generation path can be set here

2.5.3.2.3 Selection of components

To add the MCAL plugins to the project, the following steps must be done:

- Click right on your project and choose New → Other
- Choose EB AUTOSAR Tooling → Module Configuration
- Choose one plugin which shall be used
- Choose a package path

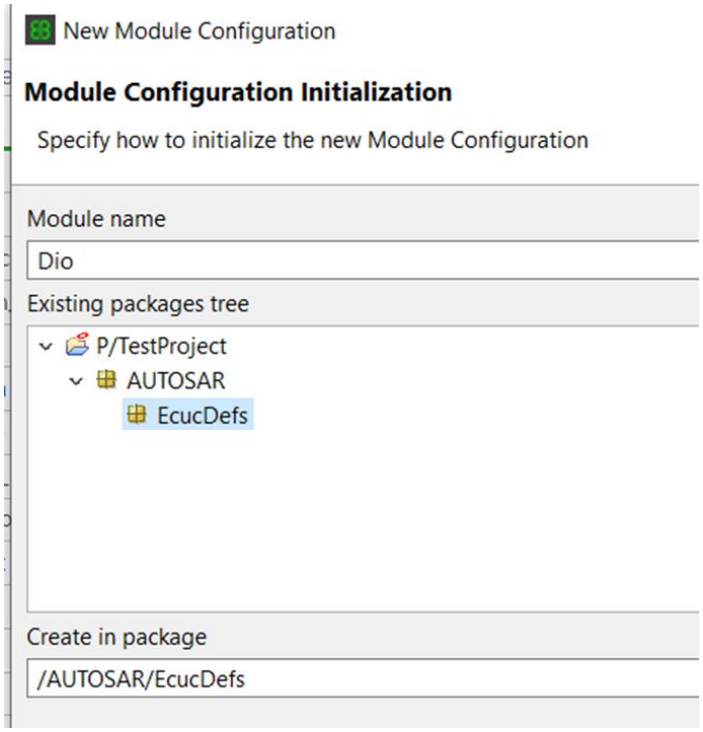


Figure 2-22 EB tresos™ User Interface: Configuration of Package Path

- Set the Module Configuration location on the next screen. All modules should be saved in the same location.

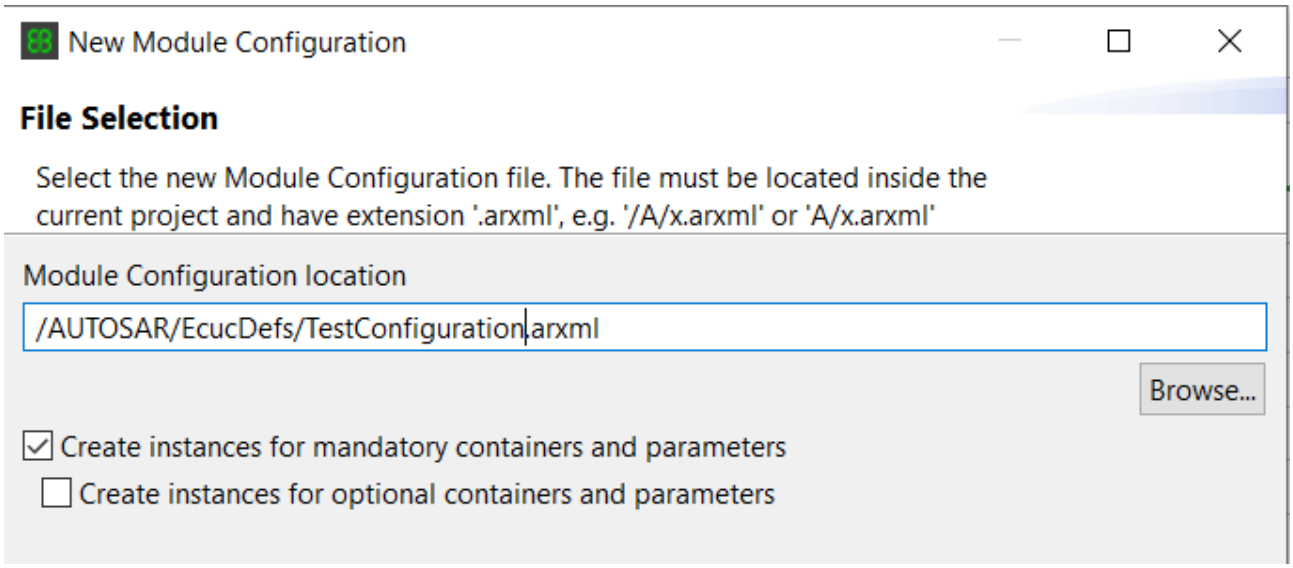


Figure 2-23 EB tresos™ User Interface: Setting the Module Configuration Location

These steps must be repeated for all modules which shall be used.

Note
To work in combination with DaVinci Configurator Classic, all modules must have the same package path and Module Configuration location.

2.5.3.2.4 Configure and Generate the MCAL Components

Now, the MCAL can be configured and generated.

2.5.4 Cooperation Interface Between DaVinci Configurator Classic and EB tresos™

With the MICROSAR Classic MCAL integration package the BSW Package is prepared for cooperation between DaVinci Configurator Classic and EB tresos™ according the recommended workflow (refer to 2.4.2). Therefore, there is no necessity for additional actions.

Nevertheless, some settings that are relevant for EB tresos™ can be configured in the DaVinci Configurator Classic Project Settings editor for Generator User Settings. Depending on the tool version it looks like follows:

2.5.4.1 EB tresos™ Classic

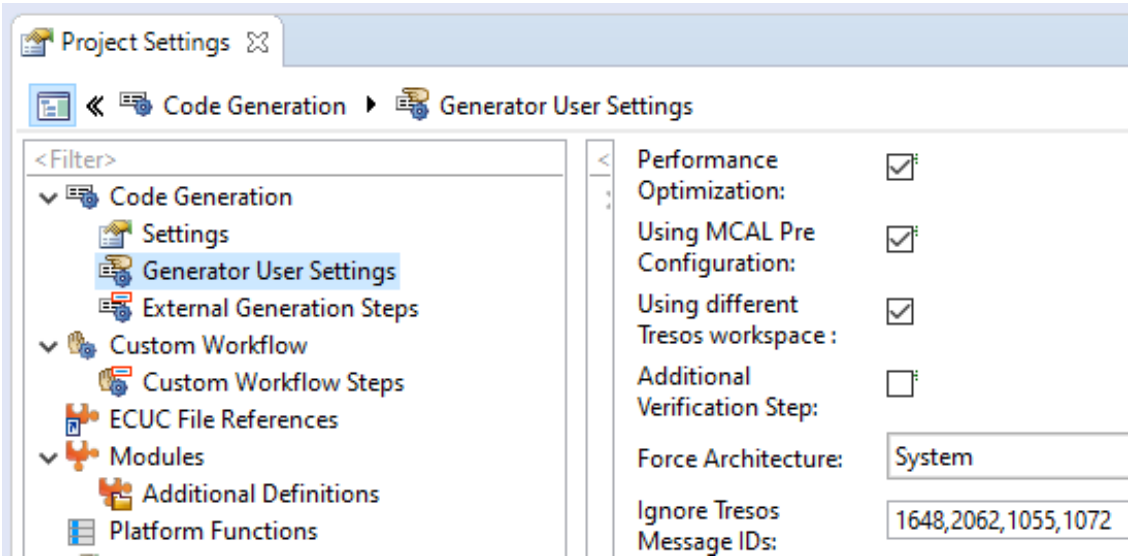


Figure 2-24 DaVinci Configurator Classic: Project Settings - Generator User Settings

The following configuration possibilities influence the behavior of the external generation tool:

- Performance Optimization:
  This switch reduces the time needed for the generation process. This can be reached by calling the external generation tool only once for all selected MCAL modules.
  Please consider that there are some restrictions when using the optimization switch:
  - The validation results for the single generated modules cannot be displayed separately, but arranged in one container named *Tresos…*:

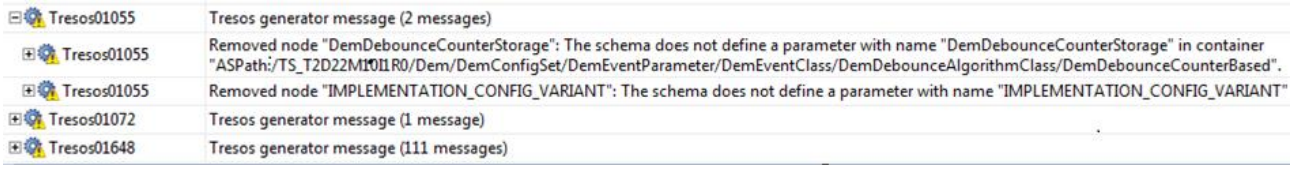


Figure 2-25 DaVinci Configurator Classic: Routing of EB tresos™ Generator Messages

  - The Generation Result View does no longer show the single generated files, but only the output folder:

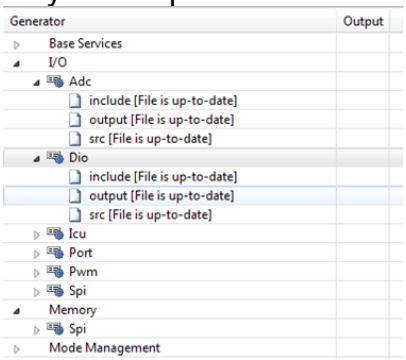
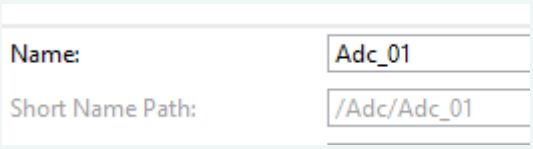


Figure 2-26 DaVinci Configurator Classic: Generation Result View

Caution
If a plug-in is renamed, e.g. from Adc to Adc_01as shown in the example, the generation without performance optimization is not possible anymore.



- Using MCAL Pre Configuration:
  Some MCALs provide pre-configuration files with unchangeable configuration parameters. The switch enables the usage of such files for the command line generation. This can only be used in combination with “Performance Optimization”. The button is only visible if the usage of the preconfiguration has an advantage. The information if this has an advantage in the concrete integration package can be found in "TechnicalReference_3rdParty-MCAL-Integration_<controller>.pdf" [4].
- Using different Tresos workspace:
  If enabled, the workspace 'WorkspaceCfg5' is created and used for MCAL generation instead of the EB tresos default workspace. This allows to generate MCAL in DaVinci Configurator Classic even if EB tresos is opened.
- Additional Verification Step:
  This switch activates a verification call of the external generation tool before generating the modules. Please consider that this leads to a longer time needed for the MCAL generation process.
- Force Architecture:
  These drop-down entries set the execution architecture of EB tresos™ (32 bit or 64 bit) manually. By default, the system property is evaluated. Further valid values are 'x86' (means 32 bit) and 'x64' (means 64 bit).
- Ignore EB tresos™ Message IDs:
  This setting disables printing of any messages with a specific message ID. Several IDs can be passed comma separated. This is meant to suppress the (sometimes) huge amount of not relevant warnings during the generation process. Please use this setting carefully.

2.5.4.2 EB tresos™ Next Generation

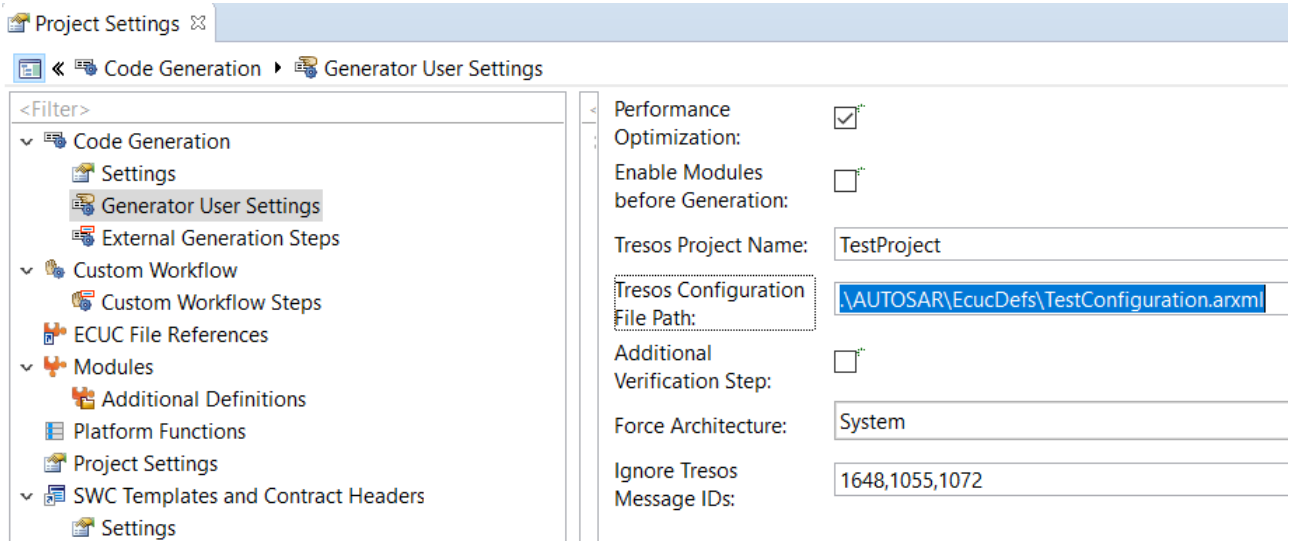


Figure 2-27 DaVinci Configurator Classic: Project Settings - Generator User Settings

> Performance Optimization:
This switch reduces the time needed for the generation process. This can be reached by calling the external generation tool only once for all selected MCAL modules.

Note
If Performance Optimization is enabled, all generators, which are enabled in tresos project, are generated independent of the chosen generators in DaVinci Configurator Classic.

> Enable modules before generation:

This setting switches on the Modules and Generators in an existing tresos project (see chapter 2.5.3.2.3)

Note
This feature should only be used if Performance Optimization is disabled and only for single MCAL modules.

> Tresos Project Name:

The name of the tresos project given in chapter 2.5.3.2.1 must be added here.

> Tresos Configuration file path:

The relative path to the configuration file given in chapter 2.5.3.2.3 must be added here.

> Additional Verification Step, Force Architecture and Ignore EB tresos™ Message IDs are currently not implemented for EB tresos™ Next generation and have no function.

2.5.5 Configuration Hints for Parallel Usage of DaVinci Configurator Classic and EB tresos™

One important thing to consider is that the configurations for Dem and EcuM must be consistent between DaVinci Configurator Classic and EB tresos™ as these are interface components provided by Vector that also have impact on the third party software. Affected configuration entities are e.g. `Dem/DemConfigSet/DemEventParameter` and `EcuM/EcuMConfiguration/EcuMCommonConfiguration/EcuMWakeupSource`.

The following chapters handle the configuration direction from upper layer to lower layer like mentioned above, but should be aware that the direction might also be inverse, i.e. for configuration of *Dio Ports*, *Mcu ConfigSets*, *Mcu ClockReferences* or *Fee Blocks*. If you use the workflows described in 2.4.3 and 2.4.2, this is done by export and import features.

2.5.5.1 Vector DaVinci Configurator Classic

You can find the configuration of e.g. `DemEventParameter` in the Basic Editor.

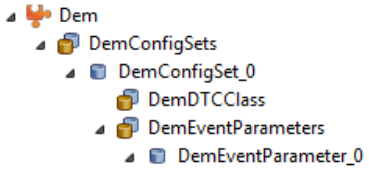


Figure 2-28 DaVinci Configurator Classic: DEM Path Using Basic Editor

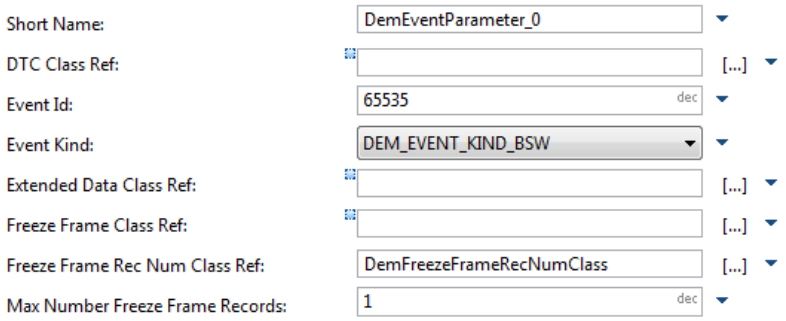


Figure 2-29 DaVinci Configurator Classic: DEM Settings

2.5.5.2 EB tresos™

> One fast way to change the configuration for DEM in tresos™ is with the Outline window.

> To do so, double-click on DEM. Now, if you have Outline opened, you will see the folder structure of DEM.

> Navigate to `DemEventParameter_0`, select the same `Event ID` and `Event Kind` as in the DaVinci Configurator Classic.

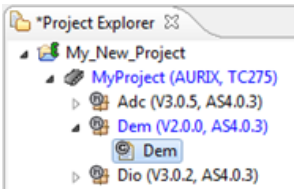


Figure 2-30 EB tresos™: DEM Module Configuration

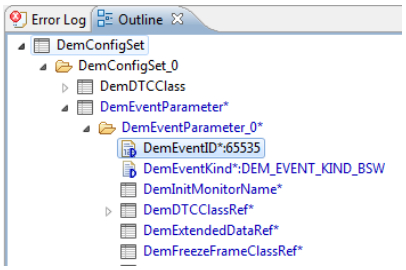


Figure 2-31 EB tresos™: DEM-Path in the Outline Window

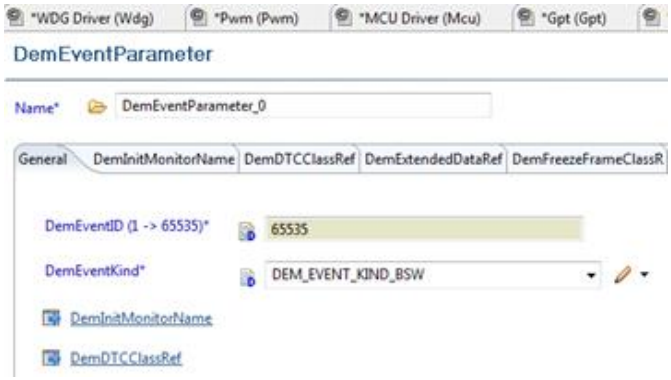


Figure 2-32 EB tresos™: Settings Within DEM Module Editor

2.5.6 DaVinci Configurator Classic Services and Comforts

2.5.6.1 Internal Behavior Editor

With the MCAL integration static values for the Internal Behavior (like Critical Sections and Main function time events) are provided. To enable the user changing and / or adding those values, a service in the User Interface exists:

Runtime System --> Module Internal Behavior

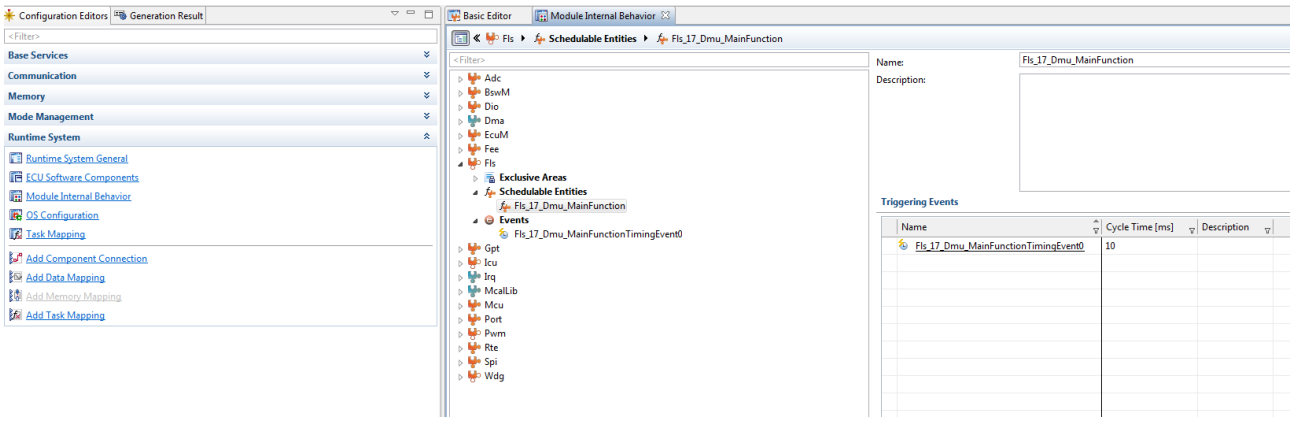


Figure 2-33 DaVinci Configurator Classic: Internal Behavior Editor

2.5.6.2 Initialization Function Service

With the MCAL integration static values for the Initialization Functions are provided. To enable the user changing and / or adding those values, a service in the User Interface exists:

Mode Management --> Initialization

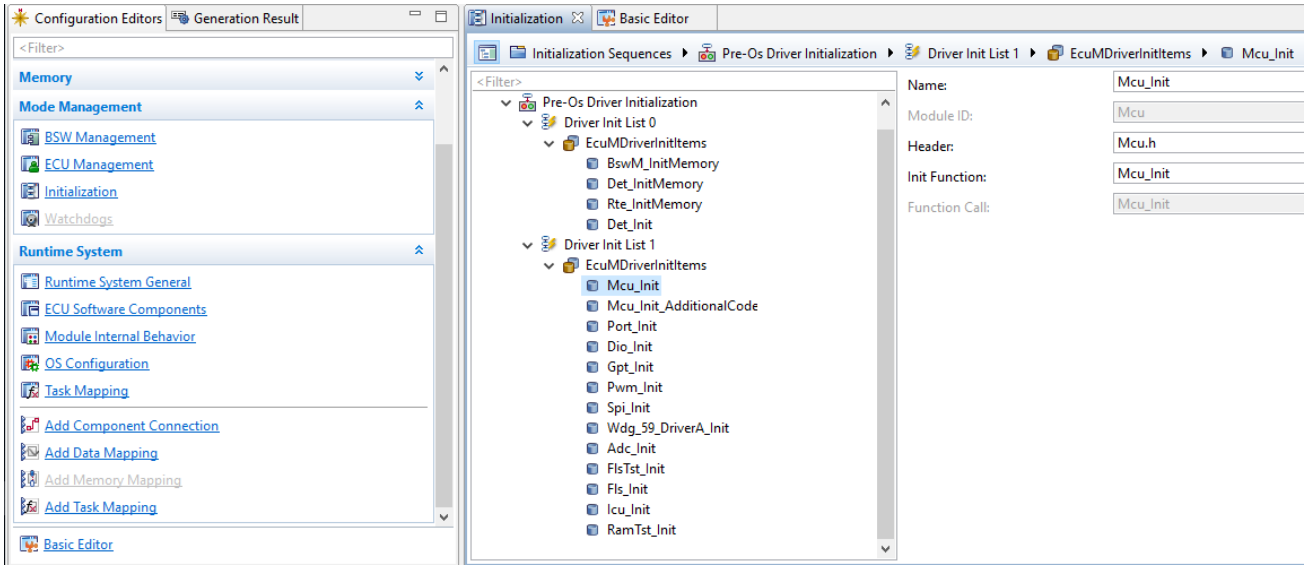


Figure 2-34 DaVinci Configurator Classic: Initialization Service

These settings are also reflected in the EcuC component:

Basic Editor → EcuC/EcucGeneral/BswInitialization/InitFunction

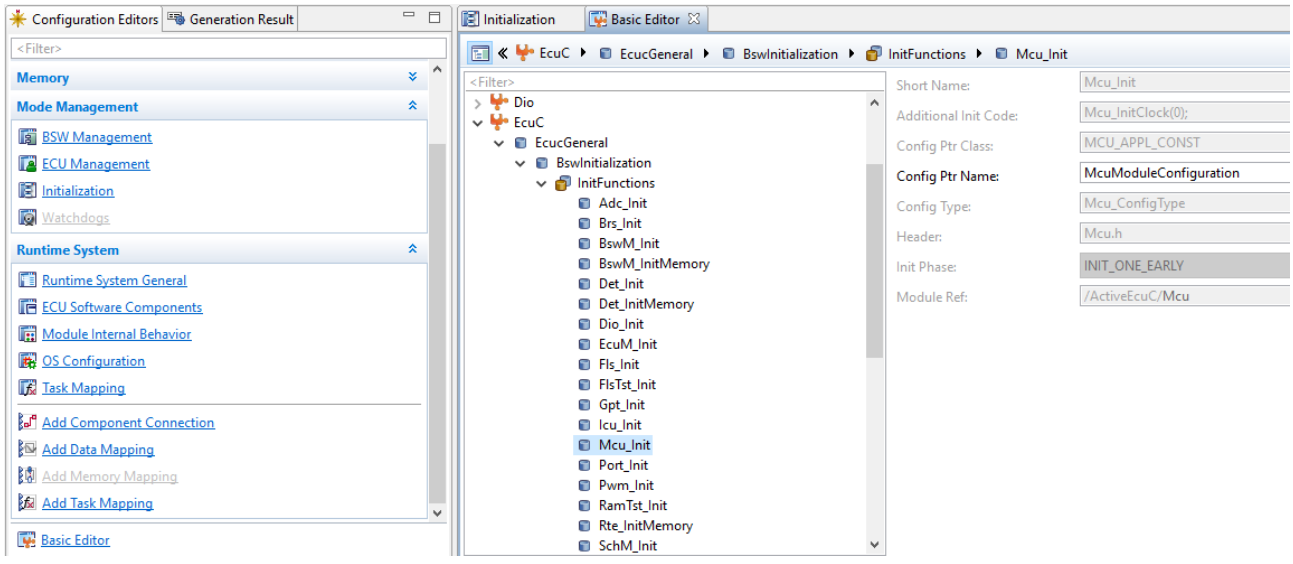


Figure 2-35 DaVinci Configurator Classic: EcuC Editor BswInitilization

3 Delivery without MCAL Integration Package

3.1 Purpose of this Chapter

This chapter provides some explanations and recommendations on how to integrate 3rdParty MCAL modules without using the Vector MCAL Integration Package.



Note
Please consider that all examples containing folder paths from the BSW Package refer to the legacy delivery structure. If your BSW Package has been delivered in component-based structure, those paths might be different.

3.2 Integration in DaVinci Configurator Classic

For the integration of a module into a MICROSAR Classic stack, different things must be done. If the module fulfills one of the following list points, check this chapter for the description.

The module:

- has parts, generated based on the configuration (i.e. ECUC file)
- requires the SCHM API for Exclusive Area handling
- has cyclic MainFunction calls
- needs access to communication PDUs

3.2.1 Configuration With DaVinci Configurator Classic

If the module shall be configured within the DaVinci Configurator Classic, the tool requires its module descriptions in a BSWMD file (Basis Software Module Description). Also, the module must be added to the configuration.

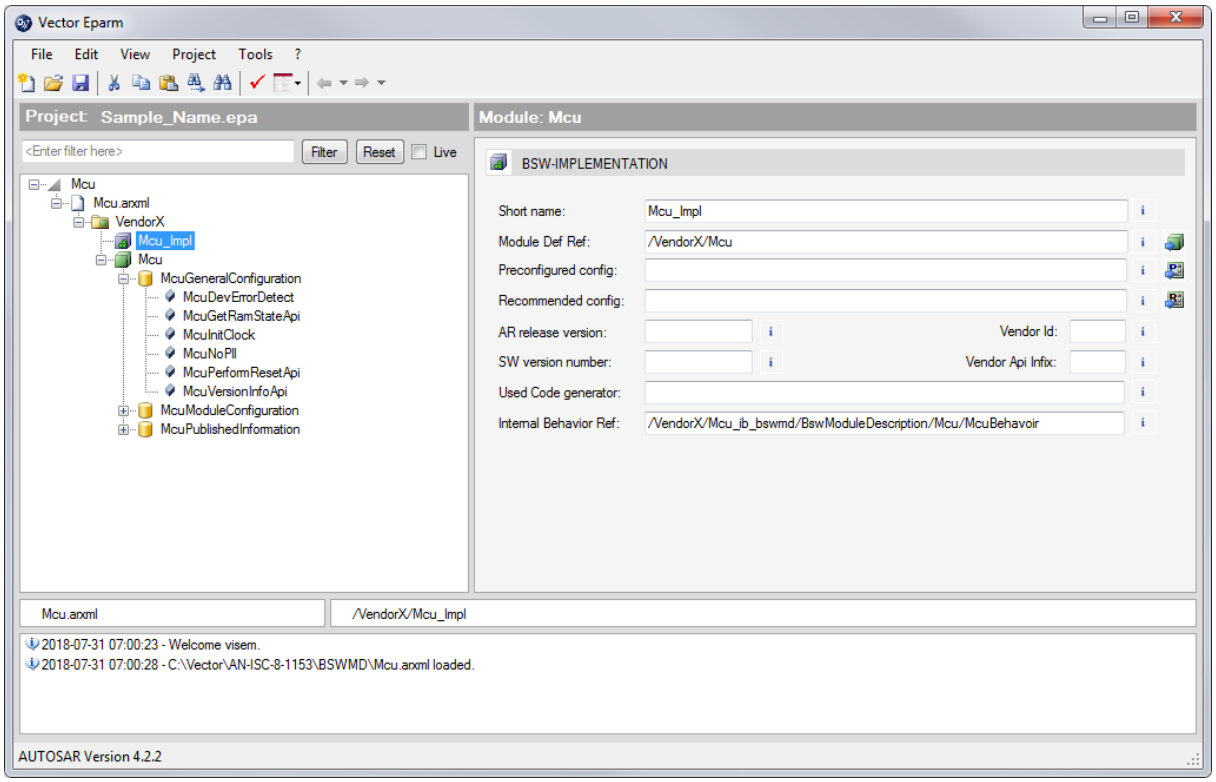


Figure 3-1 Eparm: Overview of BSWMD file of MCU

3.2.1.1 Adding of a BSWMD File

Provide an additional search path for BSWMD files within your configuration. The BSWMD file must end with '**.arxml**', '**.bmd**' or '**.epd**' to be noticed by DaVinci Configurator Classic.

Open **Project|Project Setting|Modules|Additional Definitions** and **Add** the path to the module's BSWMD file as shown in the following screenshots.

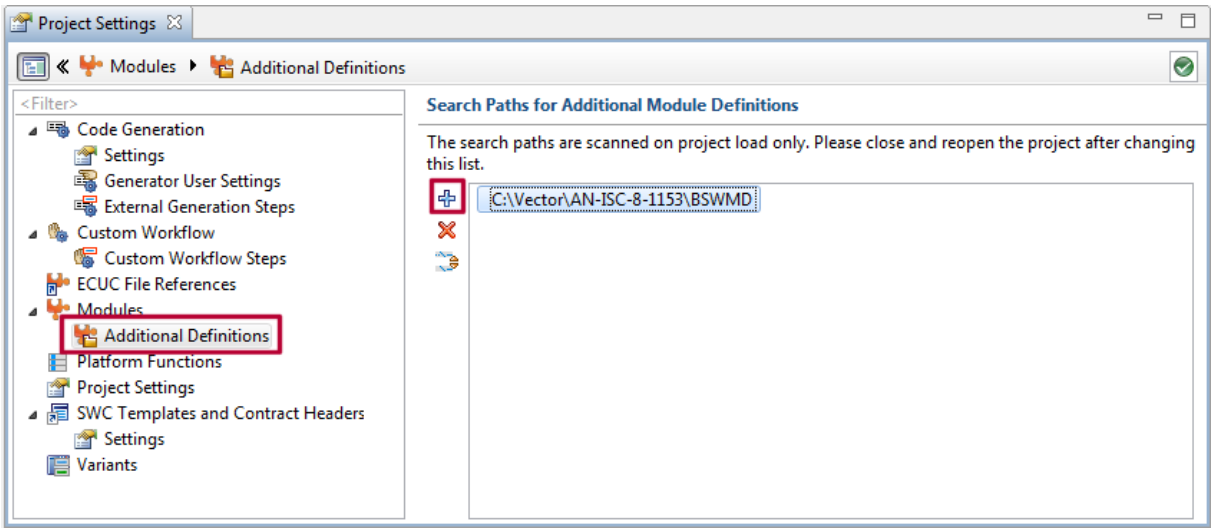


Figure 3-2 DaVinci Configurator Classic: Modules|Additional Definitions

Now DaVinci Configurator Classic knows the module and it can be added to the configuration. But before, the configuration must be closed and opened again.

3.2.1.2 Adding a Module to the Current Project

For adding the module to the current configuration, open **Project|Project Settings|Modules** and **Add** it with the blue plus. If the path to the module is within the delivered BSW Package, you will find it in **Select from SIP** otherwise in **Select additional definition** (see screenshots below).

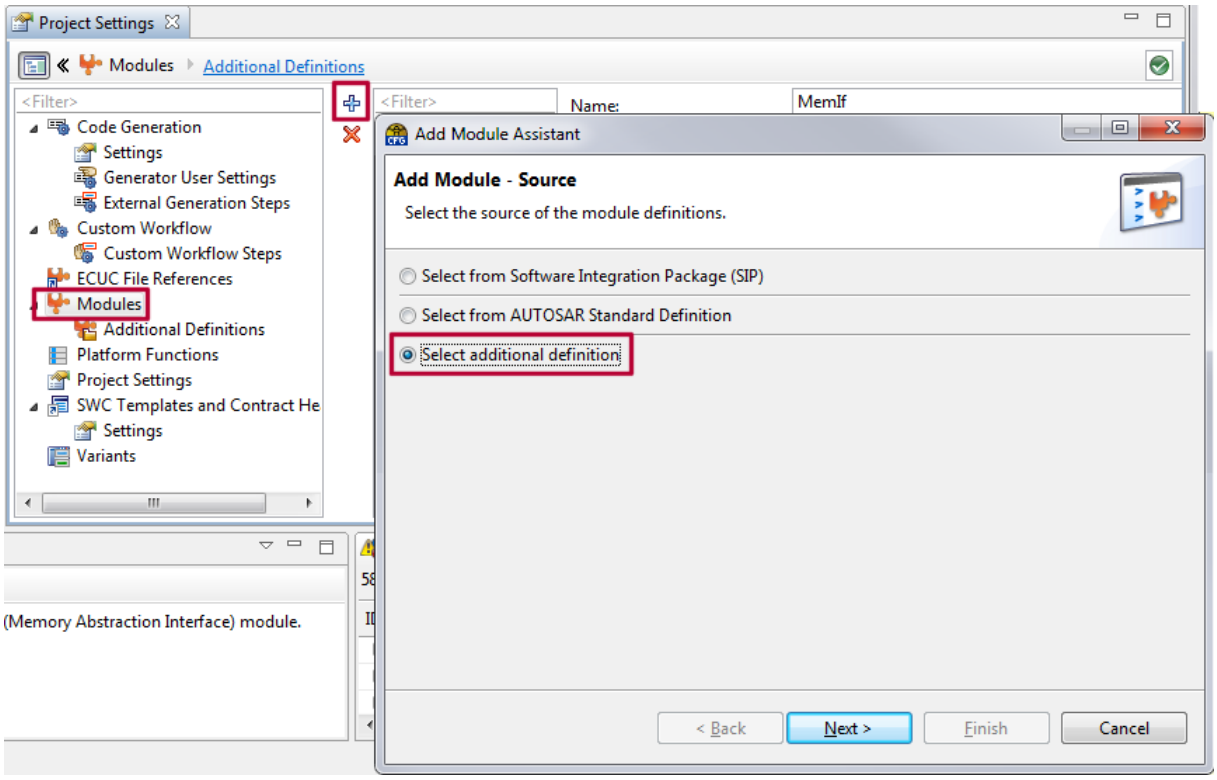


Figure 3-3 DaVinci Configurator Classic: Module Assistant

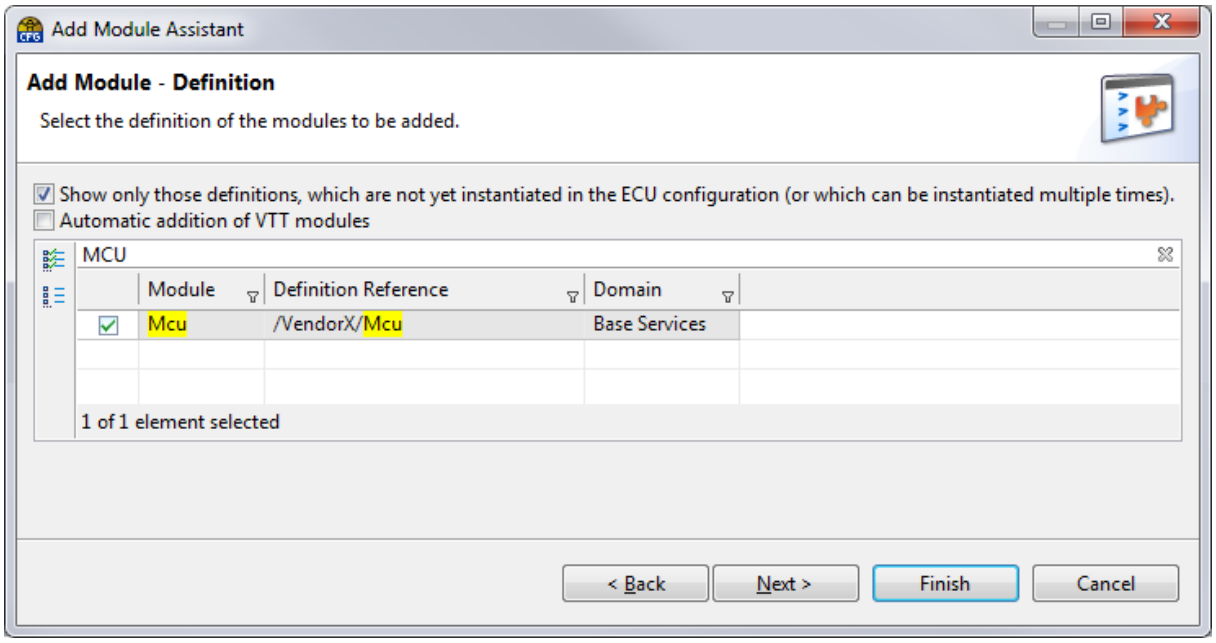


Figure 3-4 DaVinci Configurator Classic: Module Definitions

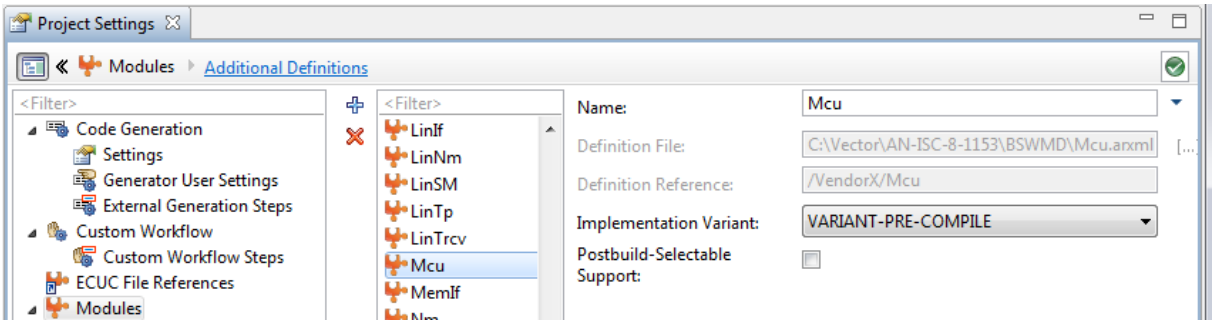


Figure 3-5 DaVinci Configurator Classic: Module is Part of the Project

The modules are now part of the project and can be configured using the Basic Editor.

3.2.2 External Generation Step

If the module has generated parts based on the configuration of the ECUC file and the generation shall be started from DaVinci Configurator Classic, the generation list must be extended. The configuration of the generation steps for 3rdParty modules can either be done manually or by a configuration file, making it easier to reuse your module for further projects.

3.2.2.1 Manual Setup

Open **Project|Project Settings|Code Generation|External Generation Steps** and **Add** the generation settings using the blue plus.

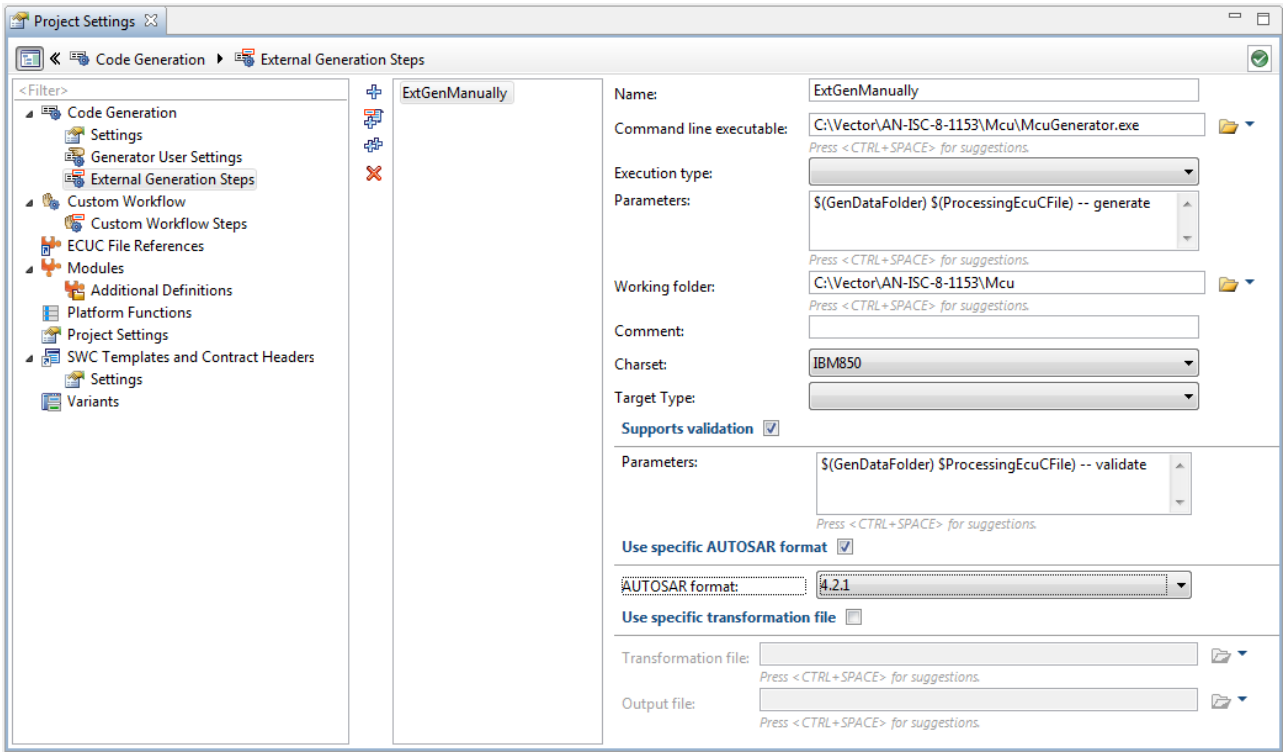


Figure 3-6 DaVinci Configurator Classic: External Generation Steps

Specify the module generator settings (i.e. parameters to be handed over; see User Manual of the External Generation Tool). If the module generator also supports validation or requires a transformation of the input file, this can also be configured. For further information also see the Help Content of DaVinci Configurator Classic.

3.2.2.2 Automatic Setup

The settings described in 3.2.2.1 can also be done automatically by a so-called **Settings.xml**. For configuration options of the Settings.xml please refer to 3.3.

3.2.3 Internal Behavior Description

Most AUTOSAR modules require **Exclusive Area** and / or **Main Function** handling by the RTE. The MICROSAR Classic RTE reads this information from the so-called Internal Behavior description, which is a part of the Input XML files like the BSWMD file. This file has to be provided to DaVinci Configurator Classic by placing it into the folder for Internal Behavior files (default is **./Config/InternalBehavior**).

The `<BSW-IMPLEMENTATION>` container within the Input XML files must have a reference to the Internal Behavior (i.e. `<BEHAVIOR-REF DEST="BSW-INTERNAL-BEHAVIOR">/VendorX/Mcu_ib_bswmd/BswModuleDescriptions/Mcu/McuBehavior</BEHAVIOR-REF>`). The RTE reads the Internal Behavior of the module from this file and provides a solving action to create a **RteBswModuleInstance** with this information.

3.2.3.1 Example of Internal Behavior Description

The following example of an Internal Behavior description defines an exclusive area called `MCU_EXCLUSIVE_AREA_0` and a Main Function called `Mcu_MainFunction`, called with a cycle time of `0.01` seconds. The file should be placed into the folder for Internal Behavior files (default is ./Config/InternalBehavior) or the content can even be in the BSWMD file itself.



Example

```
<?xml version="1.0" encoding="UTF-8" standalone="no"?>
<AUTOSAR                                          xmlns="http://autosar.org/schema/r4.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://autosar.org/schema/r4.0 AUTOSAR_4-2-2.xsd">
  <AR-PACKAGES>
    <AR-PACKAGE>
      <SHORT-NAME>VendorX</SHORT-NAME>
      <AR-PACKAGES>
        <AR-PACKAGE>
          <SHORT-NAME>Mcu_ib_bswmd</SHORT-NAME>
          <AR-PACKAGES>
            <AR-PACKAGE>
              <SHORT-NAME>BswModuleDescriptions</SHORT-NAME>
              <ELEMENTS>
                <BSW-IMPLEMENTATION>
                  <SHORT-NAME>Mcu Impl</SHORT-NAME>
                  <BEHAVIOR-REF                                          DEST="BSW-INTERNAL-
BEHAVIOR">/VendorX/Mcu_ib_bswmd/BswModuleDescriptions/Mcu/McuBehavior</BEHAVIOR-REF>
                  <VENDOR-SPECIFIC-MODULE-DEF-REFS>
                    <VENDOR-SPECIFIC-MODULE-DEF-REF                      DEST="ECUC-MODULE-
DEF">/VendorX/Mcu</VENDOR-SPECIFIC-MODULE-DEF-REF>
                  </VENDOR-SPECIFIC-MODULE-DEF-REFS>
                </BSW-IMPLEMENTATION>
                <BSW-MODULE-DESCRIPTION>
                  <SHORT-NAME>Mcu</SHORT-NAME>
                  <PROVIDED-ENTRYS>
                    <BSW-MODULE-ENTRY-REF-CONDITIONAL>
                      <BSW-MODULE-ENTRY-REF                               DEST="BSW-MODULE-
ENTRY">/VendorX/Mcu_ib_bswmd/BswModuleDescriptions/Mcu_MainFunction</BSW-MODULE-ENTRY-
REF>
                    </BSW-MODULE-ENTRY-REF-CONDITIONAL>
                  </PROVIDED-ENTRYS>
                  <INTERNAL-BEHAVIORS>
                    <BSW-INTERNAL-BEHAVIOR>
                      <SHORT-NAME>McuBehavior</SHORT-NAME>
                      <ANNOTATIONS>
                        <ANNOTATION>
                          <LABEL>
                            <L-4 L="FOR-ALL">Created</L-4>
                          </LABEL>
                          <ANNOTATION-ORIGIN>DV:IB_ED</ANNOTATION-ORIGIN>
                        </ANNOTATION>
                        <ANNOTATION>
                          <LABEL>
                            <L-4 L="FOR-ALL">Children_Renamed</L-4>
                          </LABEL>
                          <ANNOTATION-ORIGIN>DV:IB_ED</ANNOTATION-ORIGIN>
                          <ANNOTATION-TEXT>
                            <P>
                              <L-1 L="FOR-ALL">PLAIN_TEXT</L-1>
                            </P>
                            <P>
                              <L-1 L="FOR-
ALL">ExclusiveArea,MCU_EXCLUSIVE_AREA_0;SchedulableEntity,Mcu_MainFunction</L-1>
                            </P>
                          </ANNOTATION-TEXT>
                        </ANNOTATION>
                      </ANNOTATIONS>
                      <EXCLUSIVE-AREAS>
                        <EXCLUSIVE-AREA>
                          <SHORT-NAME>MCU_EXCLUSIVE_AREA_0</SHORT-NAME>
                          <ANNOTATIONS>
                            <ANNOTATION>
                              <LABEL>
                                <L-4 L="FOR-ALL">Created</L-4>
                              </LABEL>
                              <ANNOTATION-ORIGIN>DV:IB_ED</ANNOTATION-ORIGIN>
                            </ANNOTATION>
                            <ANNOTATION>
                              <LABEL>
                                <L-4 L="FOR-ALL">Renamed</L-4>
                              </LABEL>
                              <ANNOTATION-ORIGIN>DV:IB_ED</ANNOTATION-ORIGIN>
                              <ANNOTATION-TEXT>
```

```
                                <P>
                                  <L-1 L="FOR-ALL">PLAIN_TEXT</L-1>
                                </P>
                                <P>
                                  <L-1 L="FOR-ALL">ExclusiveArea</L-1>
                                </P>
                              </ANNOTATION-TEXT>
                            </ANNOTATION>
                          </ANNOTATIONS>
                        </EXCLUSIVE-AREA>
                      </EXCLUSIVE-AREAS>
                      <ENTITYS>
                        <BSW-SCHEDULABLE-ENTITY>
                          <SHORT-NAME>Mcu_MainFunction</SHORT-NAME>
                          <ANNOTATIONS>
                            <ANNOTATION>
                              <LABEL>
                                <L-4 L="FOR-ALL">Created</L-4>
                              </LABEL>
                              <ANNOTATION-ORIGIN>DV:IB_ED</ANNOTATION-ORIGIN>
                            </ANNOTATION>
                            <ANNOTATION>
                              <LABEL>
                                <L-4 L="FOR-ALL">Renamed</L-4>
                              </LABEL>
                              <ANNOTATION-ORIGIN>DV:IB_ED</ANNOTATION-ORIGIN>
                              <ANNOTATION-TEXT>
                                <P>
                                  <L-1 L="FOR-ALL">PLAIN_TEXT</L-1>
                                </P>
                                <P>
                                  <L-1 L="FOR-ALL">SchedulableEntity</L-1>
                                </P>
                              </ANNOTATION-TEXT>
                            </ANNOTATION>
                          </ANNOTATIONS>
                          <IMPLEMENTED-ENTRY-REF DEST="BSW-MODULE-
ENTRY">/VendorX/Mcu_ib_bswmd/BswModuleDescriptions/Mcu_MainFunction</IMPLEMENTED-ENTRY-
REF>
                        </BSW-SCHEDULABLE-ENTITY>
                      </ENTITYS>
                      <EVENTS>
                        <BSW-TIMING-EVENT>
                          <SHORT-NAME>Mcu_MainFunctionTimingEvent0</SHORT-NAME>
                          <ANNOTATIONS>
                            <ANNOTATION>
                              <LABEL>
                                <L-4 L="FOR-ALL">Edited</L-4>
                              </LABEL>
                              <ANNOTATION-ORIGIN>DV:IB_ED</ANNOTATION-ORIGIN>
                              <ANNOTATION-TEXT>
                                <P>
                                  <L-1 L="FOR-ALL">PLAIN_TEXT</L-1>
                                </P>
                                <P>
                                  <L-1 L="FOR-ALL">period</L-1>
                                </P>
                              </ANNOTATION-TEXT>
                            </ANNOTATION>
                            <ANNOTATION>
                              <LABEL>
                                <L-4 L="FOR-ALL">Created</L-4>
                              </LABEL>
                              <ANNOTATION-ORIGIN>DV:IB_ED</ANNOTATION-ORIGIN>
                            </ANNOTATION>
                          </ANNOTATIONS>
                          <STARTS-ON-EVENT-REF DEST="BSW-SCHEDULABLE-
ENTITY">/VendorX/Mcu_ib_bswmd/BswModuleDescriptions/Mcu/McuBehavior/Mcu_MainFunction</ST
ARTS-ON-EVENT-REF>
                          <PERIOD>0.01</PERIOD>
                        </BSW-TIMING-EVENT>
                      </EVENTS>
                    </BSW-INTERNAL-BEHAVIOR>
                  </INTERNAL-BEHAVIORS>
                </BSW-MODULE-DESCRIPTION>
```

```
                <BSW-MODULE-ENTRY>
                  <SHORT-NAME>Mcu_MainFunction</SHORT-NAME>
                  <CALL-TYPE>SCHEDULED</CALL-TYPE>
                  <EXECUTION-CONTEXT>TASK</EXECUTION-CONTEXT>
                </BSW-MODULE-ENTRY>
              </ELEMENTS>
            </AR-PACKAGE>
          </AR-PACKAGES>
        </AR-PACKAGE>
      </AR-PACKAGES>
    </AR-PACKAGE>
  </AR-PACKAGES>
</AUTOSAR>
```

3.2.3.2 Templates for Internal Behavior Description

Some modules provided by a MICROSAR delivery are not developed by Vector and do not have an automatic Internal Behavior creation (i.e. MCALs). For easier integration some of them provide templates for the Internal Behavior description. If these files are available within the current delivery they are positioned at .\Misc\InternalBehaviorTemplates. Please remove the starting underscore (‘_’) marking the files as templates, copy them to the InternalBehavior folder of your project and adapt them according to your configuration (i.e. changing the cycle time of the Main Function).

3.2.4 RTE Configuration

If the Internal Behavior is configured as described in 3.2.3, the RTE will provide a solving action to automatically create the required RTE configuration. If a Main Function must be called by the RTE, perform the according **Task Mapping** afterwards. The RTE will then generate the Exclusive Area and Main Function calls as described in the Internal Behavior.

3.2.5 CDD Configuration

For your own modules that need access to any PDU within the communication stack, use the so-called CDD module (Complex Driver), which is part of the BSW Package. For adding the CDD to the current configuration, open **Project|Project Settings|Modules** and **Add** it via the blue plus.

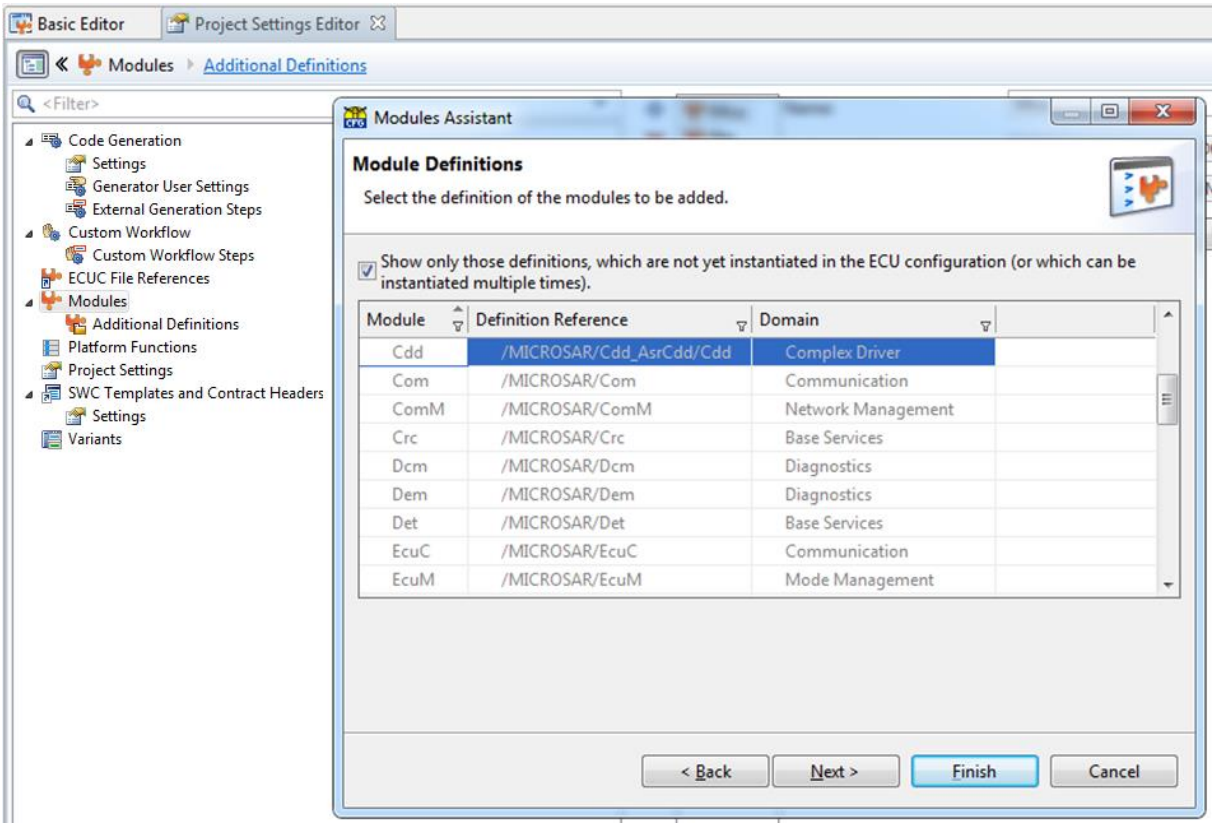


Figure 3-7 DaVinci Configurator Classic: Module Definitions CDD

In the Basic Editor you can define, where the CDD shall be placed within the communication stack.

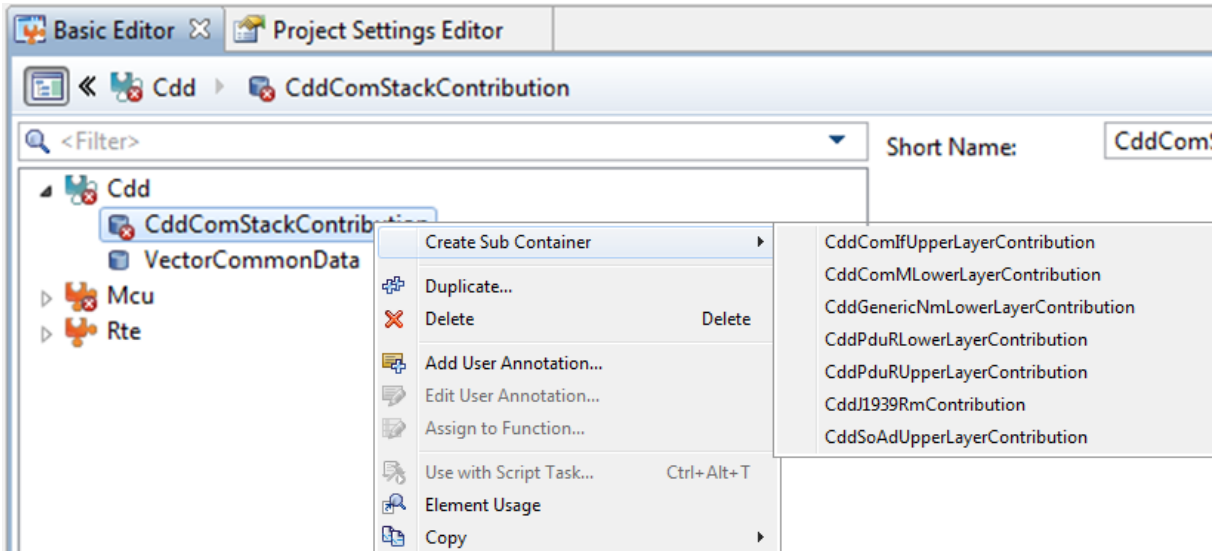


Figure 3-8 DaVinci Configurator 4: Configuring CDD

For further information how to configure the CDD, please refer to its Technical Reference [7] that is part of the BSW Package.

3.3 Settings.xml

The **Settings.xml** is an open interface to the DaVinci Configurator Classic. With this file the following configuration can be done:

- Settings for external validation and generation steps

The file shall be placed to .\DaVinciConfigurator\Generators\<Msn> (i.e. C:\Vector\<yourPath>\DaVinciConfigurator\Generators\Mcu\Settings_ExtGen_Mcu.xml) and is automatically known at start of DaVinci Configurator Classic.



Example
The following example creates the same generator settings as the example in 3.2.2.1.

```
<Settings>
  <!-- external generator -->
  <Settings Name="com.vector.cfg.gui.core.generators.ExtGenSteps">
    <Settings Name="ExtGenSettings_DrvMcu">
      <Setting Name="Active" Value="true"/>
      <Setting                                                    Name="CommandLine"
Value="C:\Vector\CBD1200333_D01_V85x_AddOn\Mcu\Mcu.exe"/>
      <Setting Name="GenerationParameters" Value="$(GenDataFolder) $(ProcessingEcuCFile)
--generate"/>
      <Setting Name="SupportsStandAloneValidation" Value="true"/>
      <Setting Name="ValidationParameters" Value="$(GenDataFolder) $(ProcessingEcuCFile)
--validate"/>
      <Setting Name="TransformationRequired" Value="false"/>
      <Setting Name="TransformationXsltFile" Value=""/>
      <Setting Name="TransformationOutput" Value=""/>
      <Setting Name="WorkingDir" Value="C:\Vector\CBD1200333_D01_V85x_AddOn\Mcu"/>
      <Setting Name="SpecificAsVersionRequired" Value="true"/>
      <Setting Name="RequiredAsVersion" Value="4.0.3"/>
    </Settings>
  </Settings>
</Settings>
```

3.4 Integration Into the Build Project

AUTOSAR has introduced a mechanism to integrate standardized code into different µC and Compilers. To adapt the modules into a project with specific compiler and linker settings the files `MemMap.h` and `Compiler_Cfg.h` have been introduced. If the module that shall be integrated into a build project that makes use of those mechanisms, these files must be adapted.

For further information on this topic please also refer to document [6] that is part of the MICROSAR Classic BSW Package.



Example code for the following chapters:

```
#define MCU_START_SEC_VAR_INIT_8BIT
#include "MemMap.h"

VAR (uint8, MCU_INIT_DATA) Mcu_InitStatus = 0;

#define MCU_STOP_SEC_VAR_INIT_8BIT
#include "MemMap.h"


#define MCU_START_SEC_PUBLIC_CODE
#include "MemMap.h"

FUNC(void, MCU_PUBLIC_CODE) Mcu_Init (P2CONST(Mcu_ConfigType, AUTOMATIC, MCU_APPL_CONST)
ConfigPtr)
{
…
}

#define MCU_STOP_SEC_PUBLIC_CODE
#include "MemMap.h"
```

3.4.1 Compiler_Cfg.h

Add all used compiler abstraction defines from your module to this file, even if they are defined to nothing.



Example

Required Compiler_Cfg.h content for above given example:

```
#define MCU_PUBLIC_CODE
#define MCU_APPL_CONST
#define MCU_INIT_DATA
```

3.4.2 MemMap.h

Add all used memory abstraction defines from your module to this file.



Example

Required MemMap.h content for above given example:

```
#ifdef MCU_START_SEC_VAR_INIT_8BIT
  #undef MCU_START_SEC_VAR_INIT_8BIT
  #define START_SEC_VAR_INIT_8BIT
#endif
#ifdef MCU_STOP_SEC_VAR_INIT_8BIT
  #undef MCU_STOP_SEC_VAR_INIT_8BIT
  #define STOP_SEC_VAR
#endif

#ifdef MCU_START_SEC_PUBLIC_CODE
  #undef MCU_START_SEC_PUBLIC_CODE
  #define START_SEC_CODE
#endif
#ifdef MCU_STOP_SEC_PUBLIC_CODE
  #undef MCU_STOP_SEC_PUBLIC_CODE
  #define STOP_SEC_CODE
#endif
```

4 Known Issues

This chapter describes known issues which are related to MCAL integration package and/or third party packages but are not fixable by Vector without support of 3rd or 4th parties. Vector is working on solutions in coordination with the other involved parties.

4.1 Long Path Names



Caution
The third party deliverables are currently stored / copied into the Vector delivery structure and may use very long path names. The Vector delivery structure itself usually resides some folders below the root directory. It may happen that some Microsoft Windows™ tools are not able to access those files anymore due to resulting path names which are too long.

Vector tools usually can deal with the limitation.



Workaround
If you recognize issues that might be caused by this restriction, we recommend installing the Vector BSW Package as well as the third party MCAL as near to the root-directory as possible (e.g. `D:\<ECU-Acronym>` like `D:\BCM`) or using the DOS SUBST command.

4.2 Missing Configuration Items Within Imported Configuration

Sometimes it happens that third party tools do not export all configuration items / information into AUTOSAR standard format of ECUC.

Currently there is no workaround available for this. The user will be informed within the import process into DaVinci Configurator Classic. After importing there must be manual editing of the configuration to add all of those “missed information” (Note: mainly these are not used by third party tooling but relevant for consistent configuration with components BSWMDs).

Note
Due to the fact that third party supply must be AUTOSAR conform please ask your MCAL vendor to provide an update of the corresponding artifacts.

4.3 EB tresos™ Project Cannot Be Created

When trying to create a project with EB tresos™, an error occurs like this one:

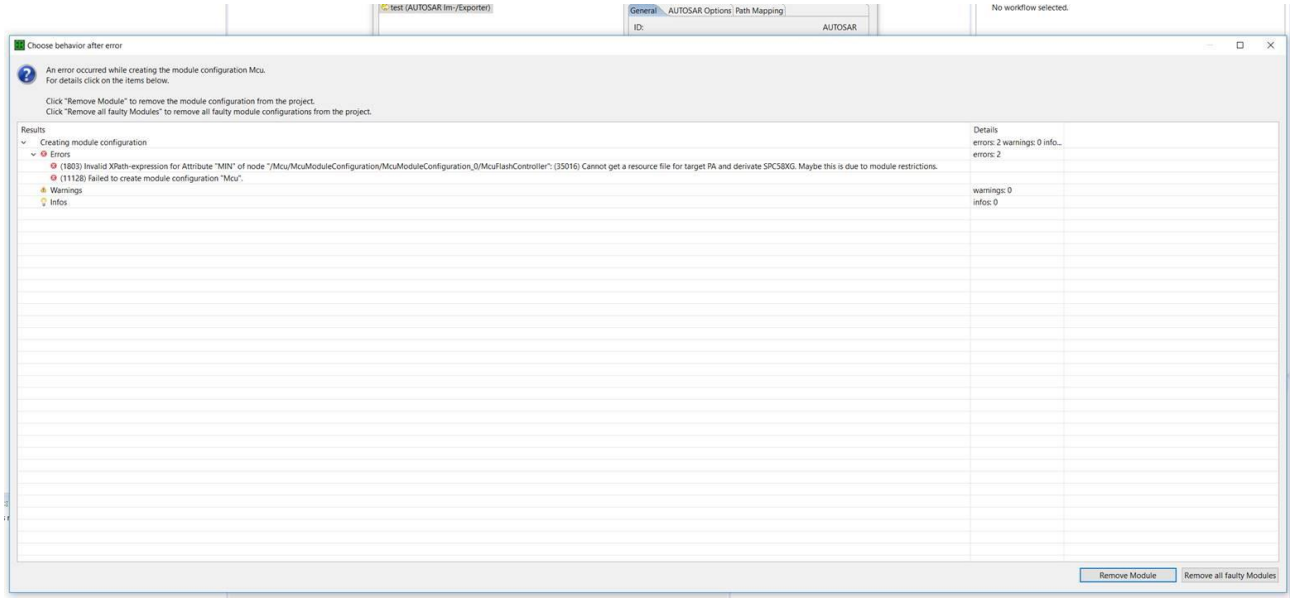


Figure 4-1 EB tresos™: Resource Restrictions

Workaround
To solve this issue, at first only add the Resource module to the project, configure the derivative parameter according your needs and save the project. Now you will be able adding the other modules without falling into this issue.

4.4 Configuration Export With EB tresos™ Version 13.0.0

After you have exported the EB tresos™ configuration more than once, the export might be faulty and cause problems during import into DaVinci Configurator Classic.



Workaround
Close the EB tresos™ project before you proceed the export the next time.

4.5 Error Messages Regarding CommonPublishedInformation With EB tresos™

After setting up an MCAL project with EB tresos™ error messages might occur for each parameter of container CommonPublishedInformation like follows:

```
Invalid value for node "/AUTOSAR/TOP-LEVEL-
PACKAGES/Wdg/ELEMENTS/Wdg/CommonPublishedInformation/VendorId
": Value "" is no number
```



Workaround
Closing EB tresos™ tool once and opening again resolves the problem. This issue affects all MCAL modules.

4.6 Problems Due to Resorting of Configuration Elements by EB tresos™ Exporter

In some cases, the EB tresos™ Exporter resorts configuration elements when converting the configuration created by EB tresos™ GUI into AUTOSAR EcuC format. Trying to generate then might lead to several problems like error messages from the external tool or code mismatch in the generated dynamic sources. This behavior has already been observed for different NXP PowerPC MCALs (Flash Sector List) and Infineon AURIX MCALs (Interrupt definitions).



Workaround
Try to resort the affected configuration entities by renaming them meaningfully (e.g. <entity>_00, <entity>_01 instead of <entity>_0, <entity>_1 for more than 10 instances). If this problem persists, the only reliable workaround is to create a configuration in EB tresos™ and use the EB tool for generation of the affected module.



Note
This issue is known for several EB tresos™ versions.

5 Frequently Asked Questions

Q: My delivery is a mixed version of BSW regarding AUTOSAR3 and MCAL regarding AUTOSAR4, what do I have to care for?

A: When you ordered a BSW Package from Vector with “MCAL Integration package (mixed ASR)” we already have cared about this and also provide you some embedded software wrappers (e.g. for Watchdog driver). The “integration process” at Vectors side has done all necessary topics like extensions of makefiles etc. that you as the user do not have to care about it.

Q: I have got a DaVinci Configurator Classic but my delivery is AUTOSAR3 based, are there any consequences or tasks I have to care about?

A: In regard of MCAL integration for AUTOSAR3 based products the DaVinci Configurator Classic does not care! As a hint: There may be some AUTOSAR4-based components which use DaVinci Configurator Classic framework or definitely have to be configured using DaVinci Configurator Classic, but this will be explained in detail with the respective components. But as mentioned before, this is not relevant for MCAL configuration / generation purpose.

Q: The MCAL will be generated component by component and takes a while. Is there any option to increase or speed up this generation on an EB tresos™ based MCAL?

A: Refer to chapter 2.5.4 sequence “Performance Optimization”.

Q: When generating an EB tresos™ based MCAL via DaVinci Configurator Classic generation errors might occur due to “Resource restrictions” although generation via EB tresos™ GUI ran without problem. What is the reason?

A: It seems device specific configuration information is missing. Add the module Resource to the DaVinci Configurator Classic project and configure the underlying parameter “ResourceSubderivatve” according the device in use.



Note
Also consider adding all other mandatory modules (e.g. Base).

6 Glossary and Abbreviations

6.1 Glossary

Term	Description
Third party components / MCAL	BSW components that have been provided by a company other than Vector. Vector may have integrated the software within the BSW Package but does not take over any responsibility with regard to the functionality of these components.
3rd party MCAL Integration Helper	Windows based tooling providing a GUI to support the user to integrate a third party MCAL into the Vector BSW Package.
DaVinci Configurator Classic	Vectors configuration and generation tool of the MICROSAR Classic components

6.2 Abbreviations

Abbreviation	Description
AUTOSAR	Automotive Open System Architecture
BSW	Basic Software
BSWMD	Basic Software Module Description (specified by AUTOSAR)
CDD	Complex Device Driver (from scope of AUTOSAR)
DEM	Diagnostic Event Manager (AUTOSAR BSW component)
DPA	DaVinci Project Assistant
ECU	Electronic Control Unit
ECUC	ECU Configuration; configuration of all BSW as well as MCAL components (Note: AUTOSAR also supports ECUC splits containing just a subset of components with its configuration)
ECUM	ECU State Manager (AUTOSAR BSW component)
(G)UI	(Graphical) User Interface
MCAL	Microcontroller Abstraction layer
MICROSAR Classic	Vector’s solution / brand for AUTOSAR BSW
RTE	Real Time Environment (AUTOSAR BSW component)
BSW Package	Formerly known as SIP or Software Integration Package (as provided by Vector); MICROSAR Classic delivery containing customers’ selection of MICROSAR Classic BSW components
SIP	s. BSW Package
StMD	Standard Module Definition also known as BSWMD (specified by AUTOSAR)
VSMD	Vendor Specific Module Definition (within AUTOSAR methodology it is derived from StMD)

7 Contact

Visit our website for more information on

- News
- Products
- Demo software
- Support
- Training data
- Addresses

www.vector.com